Bible of Practical Network Defense

Marco Marino

2025/2026

Summary

1 Networks Recap

1.1 The Internet

The **Internet** is an interconnected *network of networks*. This is a **recursive definition**: multiple independent networks (companies, organizations, universities, countries, and Internet Service Providers) are interconnected to form a global communication infrastructure.
For the Internet to operate efficiently, a **hierarchical organization** is required. Hierarchy enables a clear **separation of responsibilities** and scalable delegation of tasks. The Internet functions because many autonomous systems, devices, and subnetworks cooperate in an organized manner, distributing workload across multiple layers.
We can distinguish several structural components:

- **Internet backbone**: high-capacity core infrastructure interconnecting major ISP backbones.
- **ISP backbone**: infrastructure connecting organizational networks and providing access to the global Internet.
- **Organization backbone**: connects multiple **LANs (Local Area Networks)** within an organization.
- **LAN**: connects **end systems**.

A **backbone** is a high-capacity infrastructure composed of long-distance links, often spanning hundreds or thousands of kilometers, designed to interconnect geographically distributed networks.
End systems (also called **hosts**) are devices capable of generating, processing, and receiving **data packets**. Examples include computers, servers, smartphones, and IoT devices.

Not all networks are connected to the global Internet. Some operate as **private networks**, meaning they are isolated from the public Internet. Sensitive organizations may maintain internal infrastructures that are not externally reachable.
Communication over the Internet is governed by standardized **protocols**.

Definition 1.1 (Standard)**.** *A **standard** is an agreed set of rules and technical specifications that ensures **interoperability** and compatibility between different implementations.*

Internet standards are developed by the **IETF (Internet Engineering Task Force)** and are published in documents known as **RFCs (Request for Comments)**. These organizations consist of experts who propose, discuss, analyze, and refine protocol specifications to ensure correctness and interoperability.

1.2 Internet Architecture

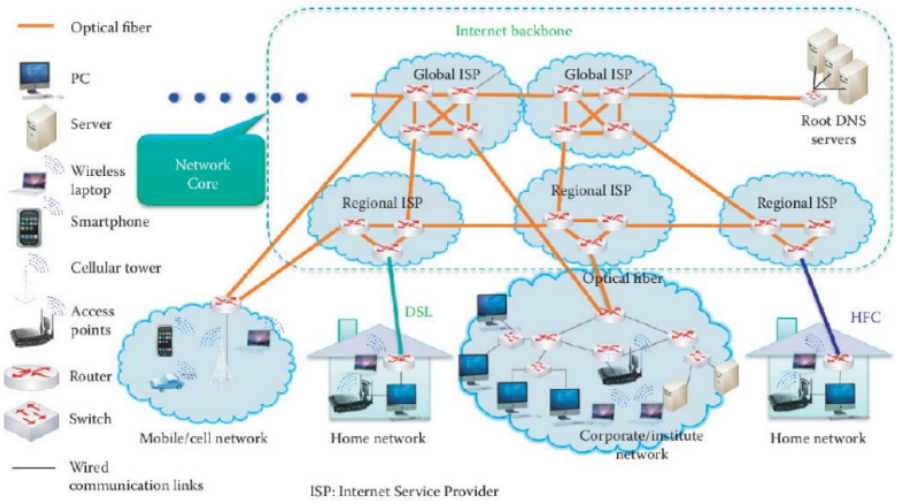


Figure 1: The Internet Architecture

The overall architecture of the Internet can be understood as a hierarchical interconnection of heterogeneous networks, as illustrated in Figure 1.
The Internet is composed of multiple interconnected components that collectively enable global communication. At the edge of the network, we find different types of access networks such as:

- **Home networks** (e.g., DSL, Fiber, Wi-Fi);
- **Corporate/Institute networks**;
- **Mobile/Cellular networks** (e.g., 4G, 5G);
- **Ethernet-based LANs**.

A **Local Area Network (LAN)** consists of devices belonging to the same administrative domain. Devices within a LAN can communicate directly without traversing the global Internet.

These access networks pay a subscription to connect to an **Internet Service Provider (ISP)** that provides:

- packet forwarding toward external networks;
- reception of packets destined to its customers;
- connectivity to higher-tier networks;
- optional service exposure (e.g., hosting public servers).

For example, a company offering online services may require a **static public IP address** to expose servers to the Internet, while a residential user typically has different connectivity requirements.

Each ISP maintains its own internal infrastructure, called the **ISP backbone**, which interconnects its points of presence and customer access networks.
When a packet is destined to a remote network, the ISP forwards it to a **next hop**, eventually reaching another ISP. This process continues until the packet reaches the destination network.

ISPs are organized hierarchically:

- **Tier-2 ISPs (Regional/National ISPs)**: purchase connectivity from higher-tier providers and provide service to local customers.
- **Tier-1 ISPs (Global ISPs)**: can reach every network on the Internet without purchasing transit from other ISPs. They interconnect with each other through peering agreements.

There are only a small number (approximately 5–10) of **Tier-1 ISPs** worldwide.
The **Internet backbone** is formed by the interconnection of Tier-1 ISP networks.

Interconnections between major ISPs often occur at **IXPs (Internet Exchange Points)**. An IXP is a physical infrastructure where multiple ISP networks interconnect and exchange traffic, reducing latency and transit costs.

The **Internet backbone** consists of the high-capacity links interconnecting regional and global ISPs. These links are predominantly based on **optical fiber**, deployed undersea, underground, and across metropolitan infrastructures.
Optical fiber provides:

- very high bandwidth;
- low latency;
- high reliability;
- immunity to electromagnetic interference.

For these reasons, fiber-optic communication is the primary technology used within the network core.

The **Root DNS servers** represent a critical global infrastructure component. They are not owned by individual ISPs but are operated by independent organizations under international coordination. Their role is to provide the top-level reference for the **Domain Name System (DNS)**, enabling global name resolution. ISPs rely on this infrastructure to provide DNS resolution services to their customers.

From a structural perspective, the Internet can be divided into three main logical areas:

- **Network Edge**;
- **Access Networks**;
- **Network Core**.

The **Network Edge** includes end hosts such as servers, clients, and peer-to-peer nodes. Applications (e.g., web services, email, messaging platforms, streaming services) run at the edge.
The **Access Network** connects end systems to their first router (typically the ISP edge router). Access technologies include wired connections (Ethernet, DSL, Fiber) and wireless technologies (Wi-Fi, 4G, 5G).
The **Network Core** is composed of high-speed routers interconnected by optical links. These routers forward packets between networks based on routing protocols. Edge routers connect organizations or access networks to the broader Internet, while core routers handle large-scale traffic forwarding across backbone infrastructures.

In enterprise network design (e.g., Cisco hierarchical model), we often distinguish:

- **Access Layer**: connects end devices.
- **Distribution Layer**: aggregates access switches and enforces policies.
- **Core Layer**: provides high-speed backbone connectivity.

Although conceptually different, this layered enterprise model mirrors the hierarchical design principles of the Internet architecture.

The Internet architecture was originally influenced by requirements emerging during the Cold War era, where the goal was to build a communication system that was **highly resilient** to failures.
Fault tolerance means that even if some nodes or links are disconnected (i.e., partial failures occur), the network should still be able to deliver packets by exploiting **alternative routes**. This is achieved through **distributed routing algorithms**, which continuously compute and update paths so that traffic can be redirected when parts of the network become unavailable.
A natural topology that supports resilience is the **mesh network**. In a mesh network, there exist **multiple distinct paths** between endpoints, so connectivity can be preserved even when some links or routers fail.

1.3 Routing

The Internet is a dynamic system: links can fail, routers can go down, and policies can change. For this reason, the **routing plane** must continuously adapt to changes in topology and reachability. Routers exchange control information and update their internal view of the network so that packets can be forwarded from a **source** to a **destination**.
This adaptability is enabled by two key elements:

- **Routing tables**: local data structures used to decide the next hop for each destination (i.e., *forwarding*).
- **Routing protocols**: distributed algorithms used to compute, maintain, and update routing information (i.e., *route computation*).

A useful analogy is **drawing vs. reading a map**:

- building/updating routing tables corresponds to **drawing the map** and is performed by **routing protocols** (e.g., **RIP**, **OSPF**, **BGP** — historically, **EGP** preceded BGP);
- using routing tables to forward packets corresponds to **reading the map**.

When forwarding a packet, routers apply a deterministic rule known as **Longest Prefix Match (LPM)**: among all routing entries that match the destination IP address, the router selects the one with the **most specific prefix** (i.e., the longest subnet mask).
Routing decisions are typically **local**: each router does *not* have a complete global view of the Internet. Instead, it relies on information learned from neighbors via routing protocols, resulting in a **partial and policy-constrained** view of reachable networks.

Finally, the "most efficient path" is not necessarily the **shortest** path in terms of hop count. Routing metrics and policies (e.g., administrative costs, link weights, traffic engineering constraints) influence path selection, and these parameters are often configured by the **network administrator** or derived from inter-domain routing policies.

1.4 Protocols and Design Principles

A **protocol** is a set of formally specified rules that define how entities in a network communicate in order to provide a service. Protocols define both the structure of exchanged messages and the behavior of the communicating parties.
A protocol is typically characterized by:

- **Procedure rules**: the type and sequence of exchanged messages, including their syntax and semantics, and the actions taken upon specific events (e.g., an HTTP GET request followed by a `200 OK` response).
- **Message format**: the structure, size, encoding, and fields of messages or packets.
- **Timing rules**: constraints on when messages are sent, retransmitted, or considered lost. For example, timeout values determine when a missing acknowledgment leads to retransmission. Timing aspects may also involve **medium access control** (e.g., in Ethernet) and **flow control** mechanisms (e.g., the sliding window in TCP), which regulate how much data can be transmitted based on network conditions and receiver capacity.

Modularization is a fundamental design principle of network architectures. Instead of implementing all functionality within a single protocol, responsibilities are divided across multiple layers and protocols.
Different protocols may provide similar services with different properties. For example:

- **TCP** and **UDP** both operate at the transport layer but offer different guarantees (reliable vs. connectionless communication).
- **DNS** can operate over either UDP or TCP depending on the context.

Protocols are designed to operate independently, interacting through well-defined interfaces. This layering allows one layer to change without affecting others. For example, a user can switch from Ethernet to Wi-Fi without modifying the web browser or application layer software.

A key architectural principle of the Internet is **packet switching**. In a packet-switched network:

- each packet is treated independently;
- packets belonging to the same flow may follow different paths;
- routers make forwarding decisions locally, based only on current routing information.

Routers do not have knowledge of the entire future path of a packet; they only determine the next hop. This decentralized decision process increases robustness and scalability.
Packet switching naturally supports a **best-effort delivery model**, where the network attempts to deliver packets without guaranteeing reliability, order, or delay bounds. Reliability and congestion control mechanisms are implemented at higher layers (e.g., TCP). This architecture enables efficient **resource sharing**, dynamic adaptation to failures, and scalable traffic management through flow and congestion control mechanisms.

1.5 Ethernet Networks (IEEE 802.3)

The **Access Layer** typically consists of networks under the same administrative control, where endpoints belong to a single local domain. Its purpose is to provide connectivity among hosts within the same local network.
The de facto standard technology for Local Area Networks (LANs) is **Ethernet (IEEE 802.3)**.

Inside a LAN, hosts can communicate directly at Layer 2 using Ethernet packets (**frames**). Frames have a fixed header format, while the payload size may vary depending on the technology.

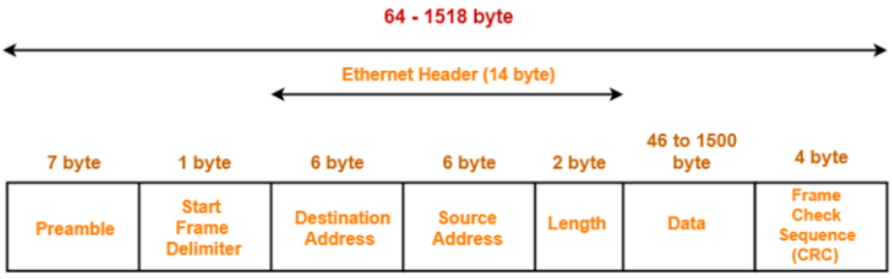


Figure 2: IEEE 802.3 Ethernet Frame Format

An Ethernet frame contains:

- **Preamble** (7 bytes) and **Start Frame Delimiter** (1 byte) used for synchronization at the physical layer;
- **Destination MAC address** (6 bytes);
- **Source MAC address** (6 bytes);

- **Length** (2 bytes): used to indicate how many bytes of payload (PDU) are stored in the frame;
- **Data** (46–1500 bytes): the actual payload transported;
- **Frame Check Sequence (CRC)**: used for error detection.

Each host connected to an Ethernet network has a **Network Interface Card (NIC)** with a unique **MAC (Medium Access Control) address**. A MAC address uniquely identifies a network interface within a broadcast domain.
If a host has multiple network interfaces, it has multiple MAC addresses (one per interface).
When a frame is transmitted, every device in the same broadcast domain can physically receive it, but only the host whose MAC address matches the destination field processes the frame; all others discard it.

Historically, Ethernet networks were implemented using a shared medium (a single cable), meaning that all devices were part of the same **collision domain**. In such networks, collisions could occur if multiple hosts transmitted simultaneously.
Modern Ethernet networks use **switches**, which provide **segmentation** of the network:

- each port of a switch represents a separate collision domain;
- frames are forwarded only to the port associated with the destination MAC address;
- broadcast frames are forwarded to all ports (except the incoming one).

Switches learn MAC-to-port associations dynamically by inspecting the **source MAC address** of incoming frames. These associations are stored in a **CAM (Content Addressable Memory) table**, while hosts store them in an **ARP (Address Resolution Protocol) table**.
This mechanism significantly reduces unnecessary traffic and eliminates collisions in full-duplex switched Ethernet.

An Ethernet network forms a **broadcast domain**. When a frame is sent to the broadcast MAC address (FF:FF:FF:FF:FF:FF), it is delivered to all hosts within the same Layer-2 domain.
However, large broadcast domains are inefficient because broadcast traffic scales poorly as the network grows.

This leads to an important design principle:

> **The Internet cannot be a single large Ethernet network.**

Ethernet is suitable for local communication within a LAN, but large-scale networks require logical segmentation and hierarchical organization.
To interconnect multiple LANs, we use **routers**, which operate at Layer 3 and forward packets based on **IP addresses** rather than MAC addresses.

- **Switches** operate within local networks (Layer 2).
- **Routers** interconnect different networks and act as default gateways.

1.5.1 MAC Addresses

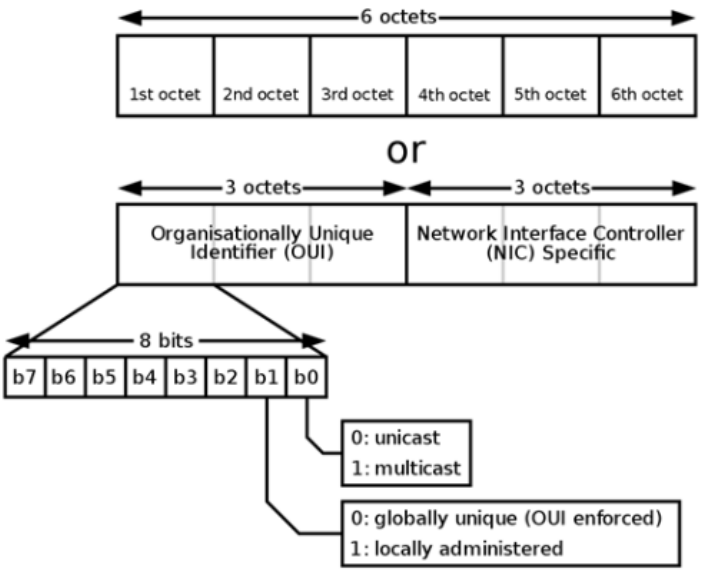


Figure 3: MAC Address Structure

A **MAC (Media Access Control) address** is a unique identifier assigned to a **network interface** (NIC) and used for communication within a Layer-2 network segment.
MAC addresses are defined by the IEEE 802 standards and are used in technologies such as Ethernet (IEEE 802.3), Wi-Fi (IEEE 802.11), and Bluetooth.

A standard MAC address is **48 bits** long (6 bytes), allowing for 2^{48} possible addresses.
The structure of a MAC address is divided into two main parts:

- The first **24 bits** (3 bytes) form the **OUI (Organizationally Unique Identifier)**, assigned by IEEE to a manufacturer.
- The last **24 bits** are assigned by the manufacturer to uniquely identify the specific network interface.

Therefore, a MAC address uniquely identifies a *network interface*, not necessarily a host (a host with multiple interfaces has multiple MAC addresses).

Within the first byte, two special bits have a specific meaning:

- The **I/G bit** (Individual/Group):
  - 0 → unicast (single destination);
  - 1 → multicast.
- The **U/L bit** (Universal/Local):
  - 0 → globally unique (assigned via OUI);
  - 1 → locally administered address.

MAC addresses are used by switches to perform Layer-2 forwarding decisions. When a frame is received, the switch examines the destination MAC address and forwards the frame only to the corresponding port (except in the case of broadcast or unknown destinations).

Although MAC addresses are intended to be globally unique, they can be manually modified (a practice known as **MAC spoofing**), which has important implications in network security.

1.5.2 IP Addresses

Two versions of the Internet Protocol are currently deployed:

- **IPv4**, introduced in 1983 within ARPANET.
- **IPv6**, standardized in the mid-1990s to address IPv4 address exhaustion.

IPv4 uses 32-bit addresses, divided into 4 octets (8 bits each), typically written in dotted decimal notation (e.g., 192.168.1.10).
The total IPv4 address space is:

$$2^{32} \approx 4.29 \times 10^9 \text{ addresses.}$$

IPv6 uses 128-bit addresses, written in hexadecimal notation, providing:

$$2^{128} \text{ possible addresses.}$$

An IP address is divided into:

- a **network prefix**;

- a **host identifier**.

All hosts within the same network share the same network prefix and differ in the host portion.
The boundary between these two parts is defined by the **subnet mask** or, in CIDR notation, by the **prefix length** (e.g., /24).

IPv4 addresses differ according to the type of communication:

- **Unicast**: one-to-one communication.
- **Broadcast**: one-to-all communication within a subnet.
- **Multicast**: one-to-many communication (to a specific group of hosts).

Recall that in IPv4, two addresses in each subnet are reserved:

- the **network address**: host bits all set to 0.
- the **broadcast address**: host bits all set to 1.

Therefore, the number of usable hosts in a subnet is:

$$2^h - 2$$

where h is the number of host bits.

In a point-to-point link (communication between exactly two hosts), using a /30 subnet provides 2 usable addresses but still reserves network and broadcast addresses. To reduce address waste, **RFC 3021** allows the use of a **/31 prefix** on IPv4 point-to-point links, eliminating the need for broadcast and network addresses in that specific context.

Originally, IPv4 used a **classful addressing scheme**, where the network/host division was fixed based on the leading bits:

Class	First Octet decimal (range)	First Octet binary (range)	IP range	Subnet Mask	Hosts per Network ID	# of networks
Class A	0 – 127	0XXXXXXX	0.0.0.0-127.255.255.255	255.0.0.0	$2^{24}-2$	2^7
Class B	128 – 191	10XXXXXX	128.0.0.0-191.255.255.255	255.255.0.0	$2^{16}-2$	2^{14}
Class C	192 – 223	110XXXXX	192.0.0.0-223.255.255.255	255.255.255.0	2^8-2	2^{21}
Class D (Multicast)	224 – 239	1110XXXX	224.0.0.0-239.255.255.255			
Class E (Experimental)	240 – 255	1111XXXX	240.0.0.0-255.255.255.255			

Figure 4: IPv4 Classes

- **Class A** (leading bit 0):
  - 8 bits network, 24 bits host
  - Number of networks: $2^7 = 128$
  - Hosts per network: $2^{24} - 2$
- **Class B** (leading bits 10):
  - 16 bits network, 16 bits host
  - Number of networks: 2^{14}
  - Hosts per network: $2^{16} - 2$
- **Class C** (leading bits 110):
  - 24 bits network, 8 bits host
  - Number of networks: 2^{21}
  - Hosts per network: $2^8 - 2$
- **Class D** (leading bits 1110): multicast addresses.
- **Class E** (leading bits 1111): experimental.

Classful addressing is now obsolete and replaced by **CIDR (Classless Inter-Domain Routing)**.

IPv4 addresses are divided into:

- **Public (routable)** addresses: globally unique and routable on the Internet.
- **Private (non-routable)** addresses: reserved for internal networks.

Private addresses ranges are defined in **RFC 1918**:

RFC 1918 name	IP address range	Number of addresses	Largest CIDR block (subnet mask)
24-bit block	10.0.0.0 – 10.255.255.255	16 777 216	10.0.0.0/8 (255.0.0.0)
20-bit block	172.16.0.0 – 172.31.255.255	1 048 576	172.16.0.0/12 (255.240.0.0)
16-bit block	192.168.0.0 – 192.168.255.255	65 536	192.168.0.0/16 (255.255.0.0)

Figure 5: Private IPv4 Addresses Range

Addresses in these ranges are not routed on the public Internet.

To allow private hosts to communicate with external networks, routers implement **NAT (Network Address Translation)**, which translates private addresses into public ones at the network boundary. NAT is used as temporary mitigation mechanism to address IPv4 address exhaustion. The long-term solution is the deployment of IPv6.

EXAMPLE 1: 192.168.5.85/24

1) IP address: 192.168.5.85

2) Subnet mask: 255.255.255.0 (/24)

3) IP binary representation: 11000000.10101000.00000101.01010101

4) Mask binary representation: 11111111.11111111.11111111.00000000

5) Bitwise AND operation: 11000000.10101000.00000101.00000000

6) Network address: 192.168.5.0

7) Broadcast address: 192.168.5.255

8) Number of host bits: $32 - 24 = 8$

9) Number of usable hosts: $2^8 - 2 = 256 - 2 = 254$

10) Valid host range: 192.168.5.1 to 192.168.5.254

EXAMPLE 2: 10.128.240.50/30

1) IP address: 10.128.240.50

2) Subnet mask: 255.255.255.252 (/30)

3) IP binary representation: 00001010.10000000.11110000.00110010

4) Mask binary representation: 11111111.11111111.11111111.11111100

5) Bitwise AND operation: 00001010.10000000.11110000.00110000

6) Network address: 10.128.240.48

7) Broadcast address: 10.128.240.51

8) Number of host bits: $32 - 30 = 2$

9) Number of usable hosts: $2^2 - 2 = 4 - 2 = 2$

10) Valid host range: 10.128.240.49 to 10.128.240.50

1.5.3 Ethernet (MAC) Addresses vs. IP Addresses

An **Ethernet (MAC) address** is a **Layer-2 physical address** associated with a specific network interface (NIC). It is typically assigned by the manufacturer and is intended to uniquely identify a network interface within a broadcast domain.
Although MAC addresses can technically be modified (a practice known as **MAC spoofing**), they are generally considered fixed identifiers bound to the hardware interface.
A MAC address does *not* change when the device moves between networks; it identifies the interface itself, not its location.

An **IP address**, instead, is a **Layer-3 logical address**. It identifies the **network location** of a device and depends on the network to which the device is currently connected. When a host moves to a different network, its IP address typically changes, because it must belong to the addressing space of the new subnet.

Conceptual difference:

- The MAC address identifies *who you are* (interface identity).
- The IP address identifies *where you are* (network location).

An intuitive analogy is the following:
If two devices are in the same local network (same broadcast domain), communication occurs directly using MAC addresses. If they are in different networks, communication requires routing at Layer 3, and therefore IP addressing is necessary to determine the correct path.
Before sending a packet, a host determines whether the destination IP belongs to the same subnet by applying the **subnet mask**.

- If the destination is in the same subnet, the frame is sent directly to the destination MAC address.
- If the destination is in a different subnet, the frame is sent to the MAC address of the **default gateway** (router).

Thus, IP addresses enable global routing across networks, while MAC addresses enable local delivery within a single Layer-2 domain.

2 Network Traffic Monitoring

2.1 Network Layering: ISO/OSI vs. TCP/IP

In order to understand what happens inside a network, we need an abstraction model. At its core, network communication consists of the movement of **packets**, which are structured sequences of bits transmitted between hosts.
For devices to correctly interpret transmitted data (i.e., where a packet starts, where it ends, and how to process it), communication must follow a shared set of rules defined by a **protocol**. Protocols specify formatting, semantics, timing, and behavior.

Modern network architectures are designed to be both **reliable** and **flexible**. Technologies and implementations may change over time (e.g., Ethernet vs. Wi-Fi, IPv4 vs. IPv6), but the overall communication model must remain consistent.
This requirement is addressed through the concept of **layering**.

In a layered architecture:

- Communication is divided into **logical layers**.
- Each layer performs a specific task.
- Each layer provides services to the layer above.
- Each layer relies on services provided by the layer below.

When a host sends data, the message is processed step-by-step by multiple layers. At each layer, additional control information may be added (a process known as **encapsulation**), modifying the structure of the data unit.
Conversely, when data is received, each layer removes its corresponding control information (a process known as **decapsulation**).

This abstraction allows:

- modular design;

- independent evolution of technologies;
- easier troubleshooting and monitoring;
- separation of responsibilities.

When data is generated at the **Application Layer**, it must traverse the protocol stack before being transmitted over the network.
At each layer, encapsulation occurs as follows:

- The layer receives a **Protocol Data Unit (PDU)** from the layer above.
- The layer adds its own **control information** (header, and sometimes trailer).
- The resulting structure is passed to the layer below.

Each layer "wraps" the data with its own protocol information, forming a nested structure.
At the sender side:

$$\text{Application Data} \rightarrow \text{Segment} \rightarrow \text{Packet} \rightarrow \text{Frame} \rightarrow \text{Bits}$$

At the receiver side, decapsulation (the reverse process) occurs: each layer removes its corresponding header and forwards the payload to the upper layer.

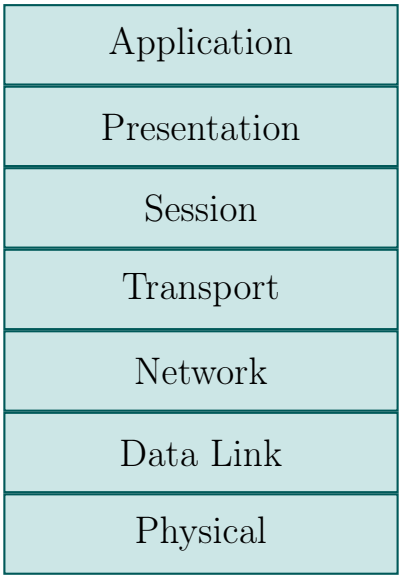
At the bottom of the stack, the **Physical Layer** converts digital information into measurable physical signals (electrical signals, light pulses in fiber, radio waves, etc.). At the receiving end, these physical signals are converted back into digital form.

It is important to note that the final destination of a packet may not be directly reachable. In many cases, packets traverse multiple intermediate devices such as **switches** and **routers**.
At each router:

- The incoming Ethernet frame is removed.
- The IP packet is inspected.
- A new frame is created for the next hop, with:
  - a new source MAC address (the router interface);
  - a new destination MAC address (the next hop).

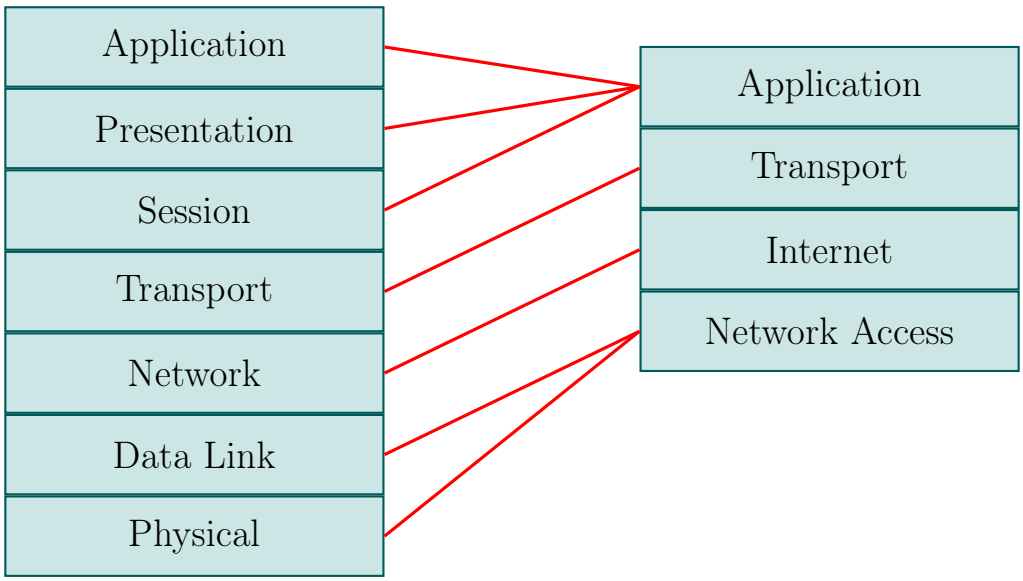
Thus, Layer-2 headers change at every hop, while the Layer-3 IP packet remains logically the same (except for fields such as TTL).

The **ISO/OSI model** is a 7-layer reference architecture:



The **Application Layer** provides network services directly to end-user applications.
The **Presentation Layer** is responsible for data representation and transformation, ensuring that data exchanged between systems is in a compatible format.
The **Session Layer** manages sessions between communicating hosts.
These three layers (Application, Presentation, Session) are often grouped conceptually.
The **Transport Layer** is responsible for end-to-end communication, multiplexing, reliability, and reordering. Protocols at this layer include **TCP** and **UDP**.
The **Network Layer** is responsible for routing and logical addressing via the **IP protocol**. Related control protocols include **ICMP**.
The **Data Link Layer** (e.g., Ethernet, PPP, Wi-Fi) handles local delivery, while the **Physical Layer** transmits bits over the medium.

The **TCP/IP model** simplifies this architecture into four layers:



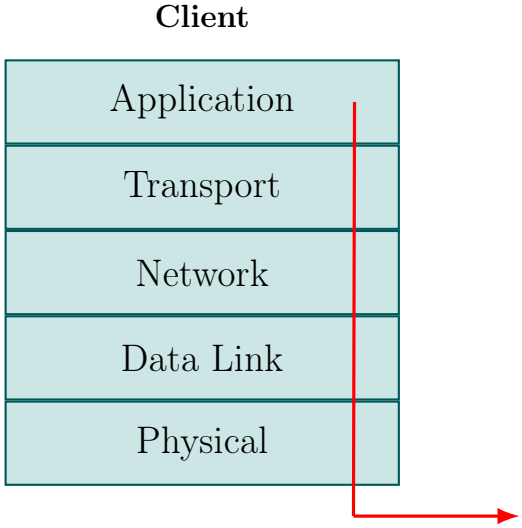
- **Application Layer** (HTTP, SMTP, FTP, DNS, etc.)
- **Transport Layer** (TCP, UDP)
- **Internet Layer** (IP, ICMP, IPsec)
- **Network Access Layer** (Ethernet, Wi-Fi, ARP + Physical layer)

Both models describe a **packet-switched network**, where each packet is processed independently and contains all necessary information to reach its destination.

2.2 Client–Server Communication

In a client–server architecture, a host (client) communicates with a remote server through the network stack.
The final goal of the communication is that the **Application layer** of the source host communicates logically with the **Application layer** of the destination host.
There is therefore a **1-to-1 logical mapping between stack layers** of source and destination.
Client–server is a step-based communication. We assume ISO/OSI model (Application, Presentation and Session Layers are grouped).

(I) When the client generates data (e.g., an HTTP request):



1. The **Application Layer**:
   - Specifies the destination host (via hostname or IP);
   - Specifies the destination port (e.g., port 80 for HTTP).
   - Passes the data to the Transport layer.

2. The **Transport Layer**:
   - Adds transport header (TCP/UDP);
   - Identifies the destination process via port number;
   - Encapsulates the data into a transport segment and passes it to the Network Layer for packet encapsulation.
3. The **Network Layer**:
   - Adds source and destination IP addresses;
   - Consults the routing table;
   - Determines whether the destination is local or remote by comparing the destination IP address with the subnet mask of the outgoing interface.

     More precisely, the Network layer computes the network prefix:

     $$(IP_{dest} \,\&\, Mask)$$

     and compares it with:

     $$(IP_{host} \,\&\, Mask).$$

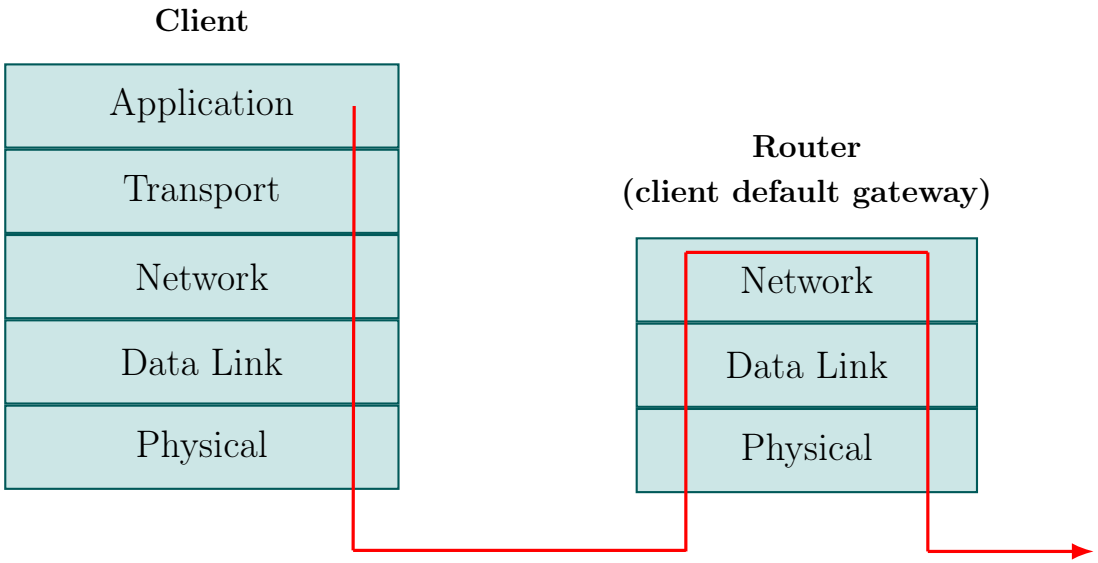
     – **If the two prefixes are equal**, the destination belongs to the same subnet (local delivery). The packet is sent directly to the destination host, and the Data Link layer will resolve the destination MAC address (e.g., via ARP).
     – **Otherwise, the destination is remote**. The Network layer selects the next hop from the routing table (typically the default gateway), and the packet is forwarded toward that router.
   - Encapsulates the transport segment into an IP packet and passes it to the Data Link Layer.
4. The **Data Link Layer**:
   - Receives the outgoing interface and next-hop information from the Network layer;
   - Adds:
     – Source MAC address (MAC of the outgoing interface);
     – Destination MAC address (MAC of the next hop, obtained via ARP if necessary).
   - Encapsulates the IP packet into a frame and passes it to the Physical Layer;

Always remember:

- The IP address usually remains the same end-to-end (except in cases in which NAT is used).
- The MAC address changes at every hop, since a new frame is created for each link.

5. At the **Physical Layer**, data is transmitted over the medium (e.g., WiFi, fiber, copper) in the form of electrical signals, optical pulses, or electromagnetic waves representing bits.

(II) When a packet reaches a router (gateway):



1. The **Physical Layer**:
   - Receives the physical signal from the medium;
   - Reconstructs the bitstream;
   - Passes the received bits to the Data Link Layer.
2. The **Data Link Layer**:
   - Reconstructs the Layer-2 frame;
   - Verifies that the destination MAC address matches one of the router's interface MAC addresses;
   - Decapsulates the frame by removing the Layer-2 header;
   - Passes the encapsulated IP packet to the Network Layer.

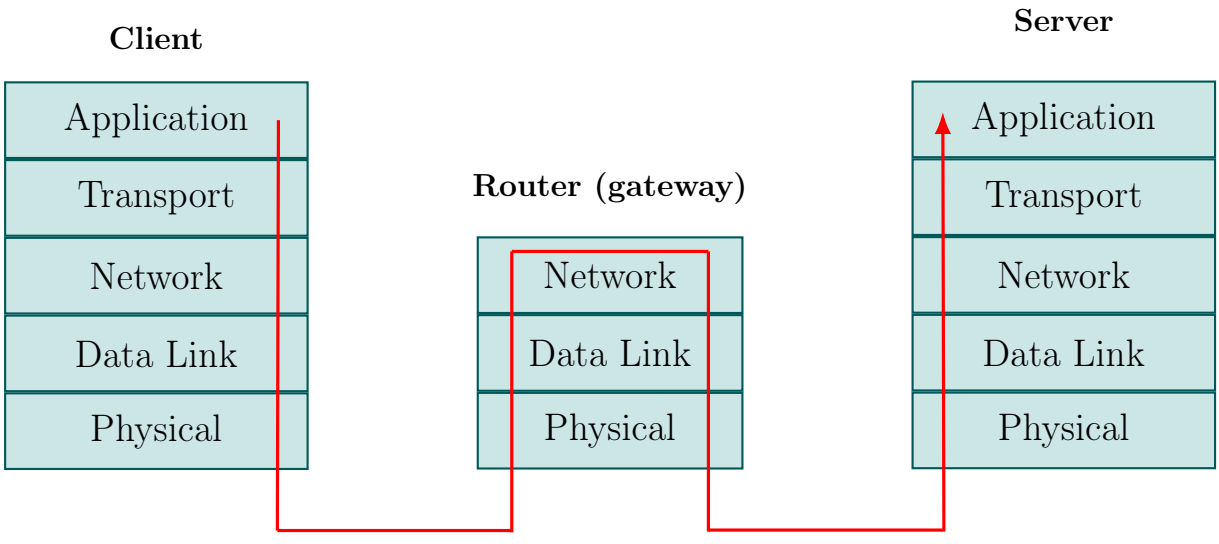
3. The **Network Layer**:
   - Reads the destination IP address from the network header;
   - Determines that the router is not the final destination (the final destination is the server);
   - Consults the routing table and performs longest-prefix matching;
   - Selects the outgoing interface and next hop: if the destination network is directly connected to one of the router's interfaces, the next hop corresponds to the final host itself; otherwise, the next hop is another router.
   - Passes the IP packet to the Data Link Layer for re-encapsulation.
4. The **Data Link Layer**:
   - Sets:
     - Source MAC = MAC of the router's outgoing interface;
     - Destination MAC = MAC of the next hop.
   - Encapsulates the IP packet into a new frame;

Thus, at every hop:

- The frame is destroyed and recreated.
- MAC addresses change.
- IP destination usually does not change, unless NAT is used.
- Payload remains unchanged.

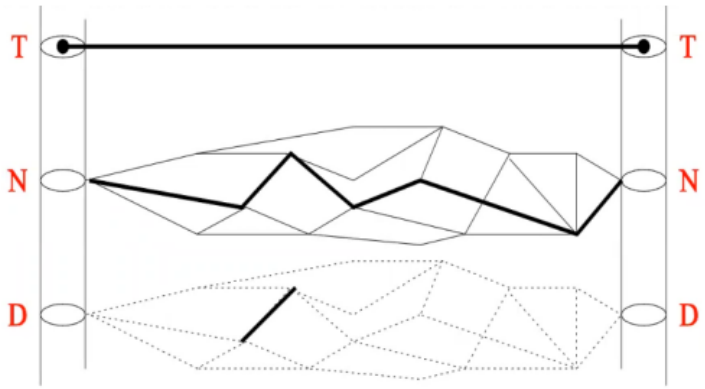
The only exception occurs **when a packet reaches a switch**, that regenerate frames and forwards them within the same local network leaving IP and MAC addresses unchanged.

(III) When a packet reaches the server:



1. The **Data Link Layer**:
   - Reconstructs the Layer-2 frame;
   - Verifies that the destination MAC address matches one of the server's interface MAC addresses;
   - Decapsulates the frame by removing the Layer-2 header;
   - Passes the encapsulated IP packet to the Network Layer.
2. The **Network Layer**:
   - Reads the destination IP address from the network header;
   - Determines that the server is the final destination;
   - Decapsulates the IP packet by removing the Layer-3 header;
   - Passes the transport segment to the Transport Layer.
3. The **Transport Layer**:
   - Reads the transport header;
   - Identifies the destination process using the port number;
   - Decapsulates the transport segment by removing the Layer-4 header;
   - Passes the application data to the Application Layer.
4. The **Application Layer**:
   - Receives the application data;
   - Delivers the payload to the appropriate application;
   - Processes the request and generates a response (if required, e.g. HTTP).

The following figure explains how different layers work:



The **Transport Layer** provides the illusion of a direct end-to-end communication channel between processes running on different hosts, without being aware of the intermediate nodes or the path taken across the network.
The **Network Layer** is responsible for transferring packets between arbitrary nodes across the network using routing mechanisms to determine the appropriate path toward the destination host.
The **Data Link Layer** operates only over a single local link transferring frames between neighboring nodes (e.g., via Ethernet or WiFi) and has no knowledge of the global network structure or the complete end-to-end path.

2.3 Characteristics and Protocols of the Layered Architecture

Each layer of the network architecture is characterized by its own **protocols** and, in many cases, its own form of **addressing**.
Different layers use different identifiers to fulfill their specific role in the communication process:

- **Application Layer**: uses **hostnames** or **Internet names** (e.g., `www.sapienza.it`), which are resolved into IP addresses via the Domain Name System (DNS).
- **Transport Layer**: uses **port numbers** to identify specific processes or services running on a host. Port numbers allow multiple applications to share the same IP address and enable multiplexing/demultiplexing of traffic.
- **Network Layer**: uses the **IP address**, which provides logical addressing and enables routing across different networks.
- **Data Link Layer**: uses the **MAC address**, which identifies a network interface within a local broadcast domain.

2.3.1 Application Layer Protocols

2.3.1.1 DNS

The **Domain Name System (DNS)** is a hierarchical and decentralized naming system used to identify devices reachable through the Internet.
Its main purpose is to translate **human-readable domain names** (e.g., `www.sapienza.it`) into **IP addresses**, which are required by network protocols to locate and communicate with hosts.

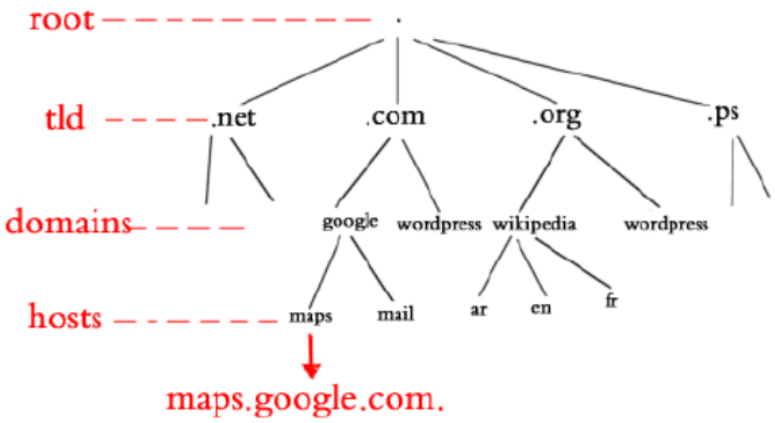


Figure 6: Tree Structure of the Domain Name Space

The DNS namespace is organized as a **tree data structure**.

- The top of the hierarchy is the **root**.
- Below the root are the **Top-Level Domains (TLDs)** (e.g., `.com`, `.org`, `.it`).
- Below TLDs are **domains**.
- Finally, we have **hosts**.

Each node in the tree contains a label and may contain one or more **Resource Records (RR)**, which store information associated with that domain name.
A complete domain name is formed by concatenating labels from the leaf node to the root, separated by dots.

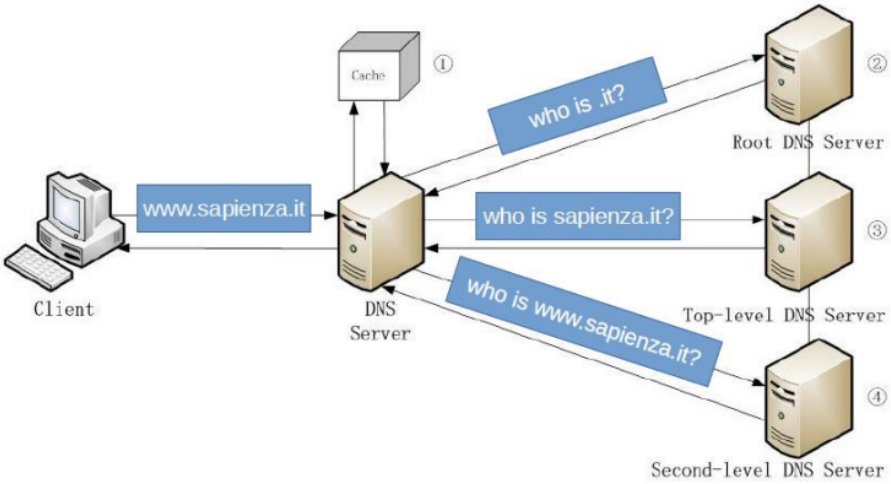


Figure 7: Recursive DNS Query

When a client needs to resolve a domain name (e.g. www.sapienza.it):

1. The client first checks its **local cache**. Each DNS record has a **Time To Live (TTL)**, which specifies how long the result can be cached before it must be requested again. Caching significantly reduces latency, network traffic and load on DNS infrastructure. If the IP address is already stored and not expired, it is reused without the need of a DNS query.

2. If not found or expired, the client sends a **recursive DNS query** to a DNS resolver (usually provided by the ISP or configured manually).

3. DNS resolvers maintain their own **DNS cache**, which stores resolved IP addresses for a limited time. If the requested IP address is stored in the cache and is still "alive", the DNS resolver returns it directly to the client without the need of other DNS queries.

4. If not found or expired, the resolver contacts the **Root DNS Server** to understand which is the DNS server that knows all the names ending with .it.

5. The Root server replies with the address of the appropriate **Top-Level Domain (TLD) server**.

6. The resolver then queries the TLD server, which replies with the address of the **Authoritative DNS Server** for the requested domain (sapienza.it).

7. The resolver then queries the authoritative server, which returns the requested IP address (www.sapienza.it).

8. Finally, the resolver returns the requested IP address to the client.

This process continues until the full domain name is resolved.

2.3.1.2 DHCP

The **Dynamic Host Configuration Protocol (DHCP)** is a client-server protocol used to automatically assign IP configuration parameters to hosts in a network.
The DHCP server maintains a **pool of IP addresses** and network configuration parameters such as:

- Subnet mask
- Default gateway
- DNS server
- Lease time

When a host connects to a network, it requests an IP address. Once assigned, the address is reserved for a limited period called the **lease time**.

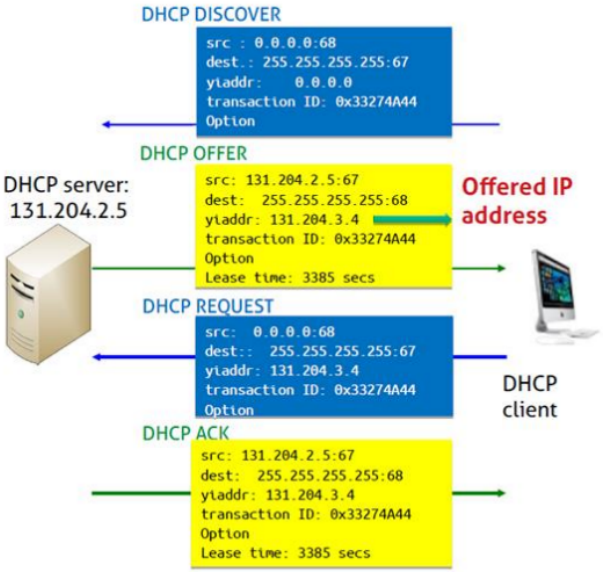


Figure 8: DHCP Procedure

The DHCP configuration process follows four main steps, often summarized as **DORA**:

1. **DHCP Discover**
   The host broadcasts a DHCP Discover message to find available DHCP servers. Since the host does not yet have an IP address:
   - Source IP address: `0.0.0.0`
   - Destination IP address: `255.255.255.255` (broadcast)
2. **DHCP Offer**
   The DHCP server responds with a DHCP Offer message containing:
   - Proposed IP address (`yiaddr`)
   - Subnet mask
   - Default gateway
   - DNS server
   - Lease time
3. **DHCP Request**
   The host broadcasts a DHCP Request message to indicate that it accepts the proposed IP address.
4. **DHCP Acknowledgment**
   The server confirms the assignment and finalizes the lease.

After the DHCP ACK, the host can start normal IP communication using the assigned configuration.

2.3.2 Transport Layer

Port numbers are 16-bit identifiers, therefore they range from 0 to 65535 (2^{16} port numbers). Ports are used at the **Transport Layer** to distinguish different services and processes running on the same host.

The **source port** is typically chosen dynamically by the operating system from the range $[49152 - 65535]$, which are the so called **ephemeral ports**. They are temporary and are used for the duration of a communication session. Their purpose is to distinguish multiple simultaneous connections initiated by the same host. The **destination port** identifies the service requested on the remote host.

Port numbers are divided into three main categories:

- **Well-known ports** (0–1023): Reserved for standard services and typically used by servers. Examples:
  - 21 – FTP
  - 22 – SSH
  - 25 – SMTP
  - 80 – HTTP
  - 143 – IMAP
  - 443 – HTTPS
- **Registered ports** (1024–49151): Assigned by the **IANA (Internet Assigned Numbers Authority)** for specific applications.
- **Dynamic/Ephemeral ports** (49152–65535): Used temporarily by clients.

A complete transport-level communication is uniquely identified by the **5-tuple**:

$$(\text{Source IP}, \text{Source Port}, \text{Destination IP}, \text{Destination Port}, \text{Protocol})$$

This tuple uniquely defines a network flow and is fundamental for:

- connection tracking,
- firewall filtering,
- NAT translation,

- network traffic monitoring.

The two main transport protocols used on the Internet are:

- **TCP (Transmission Control Protocol)**
- **UDP (User Datagram Protocol)**

Both TCP and UDP operate on top of the **IP protocol**, which provides logical addressing and routing between networks.

In most cases, an application-layer protocol is designed to use a single transport protocol. For example, HTTP typically uses TCP and SMTP uses TCP. However, there are important exceptions. Some application protocols can operate over both TCP and UDP depending on the use case. For instance:

- **DNS** can use UDP for standard queries and TCP for larger responses or zone transfers.
- **OpenVPN** can operate over either TCP or UDP, depending on configuration and performance requirements.

The choice between TCP and UDP depends on the requirements of the application, such as reliability, ordering, latency, and overhead.

2.3.2.1 UDP

User Datagram Protocol (UDP) is a **connectionless** transport protocol.
Being connectionless means that no connection is established before data transmission and no session state is maintained between sender and receiver.
UDP provides a very minimal service:

- no connection setup or teardown;
- no reliability mechanisms;
- no retransmission of lost packets;
- no flow control;
- no congestion control;
- no sequence numbers.

Each UDP segment (called a **datagram**) is sent independently of the others. If a packet is lost, duplicated, or delivered out of order, UDP does not provide any mechanism to detect or recover from these events. Any required reliability must be implemented at the application layer.

The UDP header is extremely simple and has a fixed size of **8 bytes**:

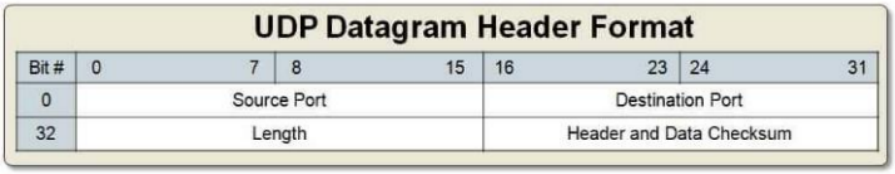


Figure 9: UDP Header

It consists of only four fields:

- **Source Port** (16 bits)
- **Destination Port** (16 bits)
- **Length** (16 bits)
- **Checksum** (16 bits)

The simplicity of the UDP header reflects the design philosophy of the protocol.

Due to its low overhead and simplicity, UDP is typically used in applications where:

- low latency is more important than reliability;
- occasional packet loss is acceptable;
- the application implements its own reliability mechanisms.

Common examples of services that rely on UDP include:

- **DNS**: port 53;
- **DHCP**: ports 67 (server) and 68 (client);
- **TFTP**: a lightweight file transfer protocol, port 69.
- **SNMP**: used for network management and monitoring, port 161.
- **RIP**: a distance-vector routing protocol, port 520.

2.3.2.2 TCP

Transmission Control Protocol (TCP) is a **connection-oriented** transport protocol that provides:

- Reliable data delivery
- Flow control
- Congestion control
- Ordered data transmission

Unlike UDP, TCP ensures that lost packets are retransmitted and that data is delivered in the correct order. Obviously, these additional guarantees come at the cost of a more complex header:

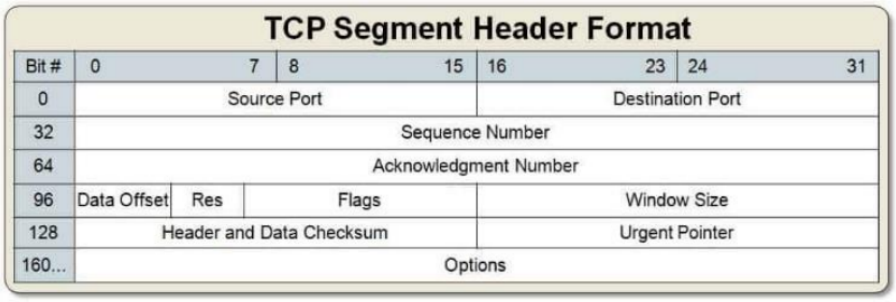


Figure 10: TCP Header

The TCP header includes fields such as:

- Sequence number (packet ordering)
- Acknowledgment number (reliable delivery)
- Window size (flow control)
- Control flags (SYN, ACK, FIN, RST, etc.)
- Checksum (error checking)

The minimum TCP header size is **20 bytes**.

Since TCP is connection-oriented, before exchanging data, two hosts must establish a connection using the so called **three-way handshake**:

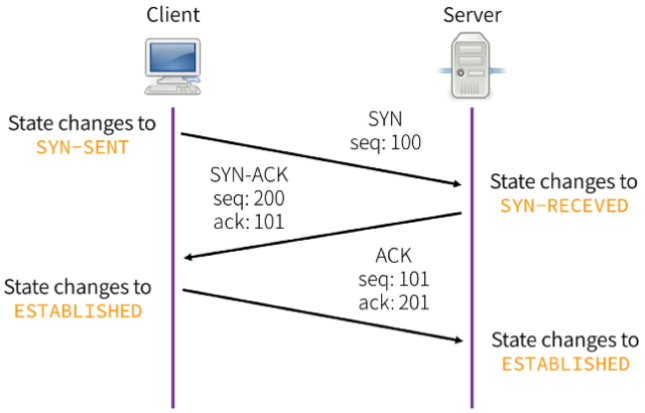


Figure 11: TCP Three-Way Handshake

This procedure involves three segments without payload (no actual data is exchanged during the procedure):

1. **SYN**:
   The client initiates the connection by sending a SYN segment with:
   - SYN flag set;
   - an initial sequence number (ISN) set to a random value x. In early TCP implementations, weak random number generators made initial sequence numbers predictable, enabling TCP session hijacking attacks. Modern implementations use secure randomization mechanisms to mitigate this vulnerability.
2. **SYN-ACK**:
   The server responds with a SYN-ACK segment with:
   - SYN flag set;
   - ACK flag set;
   - its own initial sequence number y;
   - acknowledgment number equal to $x + 1$.
3. **ACK**:
   The client replies with an ACK segment with:
   - ACK flag set;
   - Sequence number equal to $x + 1$;
   - Acknowledgment number equal to $y + 1$.

After this exchange, both client and server transition to the **ESTABLISHED** state and can start transmitting data.

Common examples of services that rely on TCP include:

- **FTP** – ports 20/21
- **SSH** – port 22
- **Telnet** – port 23
- **SMTP** – port 25
- **HTTP** – port 80
- **IMAP** – port 143
- **HTTPS (SSL/TLS)** – port 443

2.3.3 Network Layer: IPv4

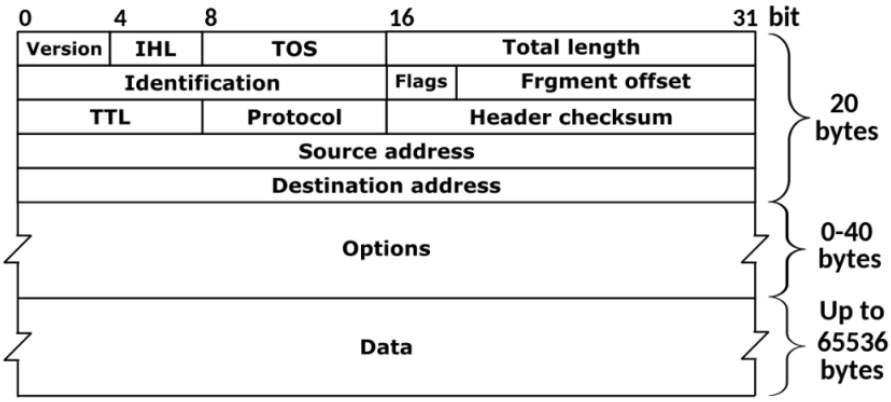


Figure 12: IPv4 Packet

An IPv4 packet consists of:

- A fixed header of **20 bytes**;
- An optional field (**Options**) of 0 to 40 bytes;
- The data payload.

Therefore, the IPv4 header size, specified by the **IHL (Internet Header Length)** field, ranges from a minimum of **20 bytes** to a maximum of **60 bytes**, excluding the payload.

The main fields include:

- **Version**: indicates the IP version (4 for IPv4).
- **IHL**: specifies the header length.

- **Type of Service (TOS)**: indicates service quality parameters. It is rarely used in basic configurations.
- **Total Length**: specifies the total size of the IP packet (header + payload).

There's also a special set of fields that manage **fragmentation**, which occurs when a packet exceeds the **MTU (Maximum Transmission Unit)** of a link:

- **Identification**: identifies fragments belonging to the same original packet.
- **Flags**: indicate whether fragmentation is allowed and whether more fragments follow.
- **Fragment Offset**: specifies the position of the fragment within the original packet.

Fragmentation has historically been a source of weaknesses in IP. IPv6 redesigns fragmentation mechanisms to reduce its use.

The **TTL (Time To Live)** field prevents packets from circulating indefinitely in the network. Each time a packet is forwarded by a router (hop), the TTL value is decreased by 1. When TTL reaches 0, the packet is discarded and the router sends an **ICMP "Time Exceeded"** message to the sender. TTL is fundamental for detecting routing loops and is used by the **traceroute** utility.

Traceroute works by sending packets with increasing TTL values:

1. The sender transmits a packet with **TTL = 1**.
   The first router decrements TTL to 0, discards the packet, and replies with an ICMP Time Exceeded message. From this reply, the sender learns the IP address of the first router.
2. The sender transmits a packet with **TTL = 2**.
   The packet reaches the second router before expiring. The second router sends an ICMP Time Exceeded message, revealing its address.
3. The process continues with TTL = 3, 4, 5, ... until the packet reaches the final destination.

By collecting the sequence of ICMP responses, traceroute reconstructs the path followed by packets across the network, hop by hop. The round-trip time (RTT) for each hop is also measured, providing insight into network latency.

The remaining fields include:

- **Protocol**: identifies the upper-layer protocol carried in the payload (e.g., TCP, UDP, ICMP).

- **Header Checksum**: verifies integrity of the IPv4 header.
- **Source Address** and **Destination Address**: logical IP addresses of sender and receiver.

2.4 Capturing Packets

To observe what happens in a network, it is possible to capture the packets that flow through the interfaces using a **network traffic dump tool**.
Several tools can be used for this purpose:

- **dumpcap**: a simpler and less advanced capture tool.
- **tcpdump**: a command-line tool that allows packet capture and supports filtering mechanisms.
- **Wireshark** and **tshark**: more advanced tools capable of visualizing packets and analyzing or decoding their internal structure.

All these tools are based on the **pcap library**.
In practice, **Wireshark** is commonly used because it provides a graphical interface that allows the user to easily inspect captured packets. However, **tcpdump** is also very important since it can be used on servers or systems without a graphical interface and is therefore a very useful tool for **network diagnostics**.

2.4.1 Wireshark

INSERT WIRESHARK SCREENSHOT!

When Wireshark captures traffic from a network interface, it analyzes the received data by identifying the different protocol layers. Captured frames are passed to a series of protocol analyzers called **dissectors**. The dissection is organized according to the protocol stack:

- **Frames** (Layer 2 – Ethernet)
- **Packets** (Layer 3 – IP)
- **Segments** (Layer 4 – TCP/UDP)

Each dissector is responsible for understanding the structure of a specific protocol. The analysis proceeds from the outer protocol to the inner one, for example:

$$\text{Ethernet} \rightarrow \text{IP} \rightarrow \text{UDP} \rightarrow \text{DNS}$$

Each layer is analyzed by a specialized dissector that interprets the fields according to the protocol specification.

Protocols can be **identified** in two main ways:

- **Direct detection**: the protocol explicitly indicates which protocol it is carrying. For example:
  - Ethernet specifies the encapsulated protocol;
  - IP indicates the upper-layer protocol;
  - TCP or UDP ports can reveal the application protocol.
- **Indirect detection**: the protocol is inferred using protocol/port tables and heuristics. This usually works well but may fail when protocols run on non-standard ports.

Wireshark is typically used for:

- identifying the root cause of a known network problem;
- inspecting specific protocols or traffic streams;
- analyzing timing, flags, or bit-level details of packets;
- following a communication flow between two hosts.

However, Wireshark is usually not the first tool used to discover a problem; it is more commonly used once the existence of a problem is already known. **Alternative approaches** to traffic monitoring include:

- **NetFlow**: a mechanism used by routers and switches to collect traffic statistics.
- **Zeek**: a framework for deep traffic inspection and intrusion detection.

Promiscuous Mode

In Ethernet networks, every host could potentially receive every frame on the network. However, the network card filters frames by checking the **destination MAC address**. A frame is delivered to the upper layers only if:

- the destination MAC address matches the host MAC address;
- the frame is a broadcast;
- the frame is a multicast that includes the host.

This filtering represents a limitation when we want to analyze all the traffic flowing in the network.
The **promiscuous mode** bypasses this limitation: all received frames are delivered to the upper layers without performing MAC address filtering.
In Wi-Fi networks, this mode is referred to as **monitor mode**. In this case, the wireless interface can capture all packets in the air, even if they belong to different SSIDs or networks.

Wireshark Filtering

Wireshark provides two types of filters:

- **Capture filters**: they define which packets are captured. Packets that do not match the filter are **not captured and are lost**. These filters are applied during the capture phase.
- **Display filters**: they allow the user to visualize only the packets that match certain conditions **after** the capture has been performed, without losing any packets. They use a richer syntax with comparison operators and logical expressions.

Capture filters use the **BPF (Berkeley Packet Filter)** syntax.
The general structure of a capture filter is:

`protocol direction type`

Examples of elements that can appear in filters:

- Protocols: `tcp`, `udp`, `ip`, `ipv6`, `ether`, `arp`
- Types: `host`, `port`, `portrange`, `net`
- Directions: `src`, `dst`

Additional primitives include:

- `less`, `greater`, `gateway`, `broadcast`

Filters can be combined using logical operators:

`and(&&)`, `or(||)`, `not(!)`

Finally, Wireshark can also filter traffic using **geographical information** via a GeoIP resolver. For example, it is possible to filter traffic originating from a specific country:

```
ip.geoip.country == "China"
```

Promiscuous Mode Limitations

Promiscuous mode works effectively only in networks using **hubs**. In switched networks, each host is connected to a dedicated switch port, and packets are delivered only to the intended destination.
Therefore, a host cannot normally see all traffic in the network.
Possible solutions include:

- using a **network tap** (hardware splitter);
- using **port mirroring** on managed switches.

In Cisco terminology, port mirroring is called:

- **SPAN** (Switched Port Analyzer)
- **RAP** (Roving Analysis Port)

In these configurations, traffic is replicated to a dedicated monitoring port.
More aggressive traffic interception techniques include:

- **ARP cache poisoning (ARP spoofing):** an attacker forges *unsolicited ARP replies* in order to poison the ARP cache of one or more hosts. By doing so, the attacker can bind **its own MAC address** to the **IP address of another host** (typically the default gateway), effectively *stealing* traffic and enabling a **Man-in-the-Middle (MitM)** position. Common tools: `ettercap`, `Cain&Abel`.
- **MAC flooding:** the attacker sends a large number of Ethernet frames with **different (spoofed) source MAC addresses** in order to **fill the CAM table** (Content Addressable Memory) of a switch. When the CAM table is exhausted, the switch may start behaving more like a *hub*, broadcasting frames on multiple ports, which makes passive sniffing easier. Common tool: `macof`.
- **DHCP redirection:** the attacker introduces a **rogue DHCP server**. A common strategy is:
  - **DHCP starvation**: exhaust the available IP pool by generating many DHCP requests, so that legitimate clients cannot obtain addresses.
  - **Gateway/DNS manipulation**: respond to new DHCP requests by advertising the attacker as the **default gateway**, redirecting traffic through the attacker.

  Common tools: `Gobbler`, `DHCPstarv`, `Yersinia`.
- **Redirection and interception with ICMP:** an attacker sends **ICMP Redirect** messages (**Type 5**) to convince a host that there is a "better" route for a destination. If accepted, the victim updates its routing decision and starts sending traffic to the attacker as the next hop, enabling traffic interception. Common tool: `ettercap`.

In **virtualized environments** or SDN networks, packet capture can sometimes be simpler (for example in environments such as Kathara).

Preventing Packet Capture

To mitigate traffic interception, switches can implement mechanisms such as:

- **Dynamic ARP Inspection (DAI)**: validates ARP packets and verifies IP-to-MAC bindings, dropping invalid packets.
- **DHCP Snooping**: distinguishes between trusted and untrusted ports and maintains a database of valid IP-to-MAC associations. Ports that show rogue activity can be placed in a disabled state.

3 Laboratories

3.1 Laboratory of 26/02/26

3.1.1 Configuring a Host

In order to properly access the Internet, a host must be configured with **four main pieces of information**:

- **IP address**
  The unique logical address that identifies the host within the network.
- **Subnet mask**
  Used to distinguish the network portion from the host portion of the IP address.
- **Default gateway IP address**
  The address of the router in the local network that provides access to external networks (distribution layer).
- **DNS server IP address**
  The address of a remote server responsible for translating human-readable domain names into IP addresses.

In Debian-based systems, network interfaces can be manually configured through:

```
/etc/network/interfaces
```

Additional configuration files can be placed in:

```
/etc/network/interfaces.d/
```

After modifying the configuration, apply changes using:

```
ifdown <interface>
ifup <interface>
```

Static Configuration Example

```
auto eth0
iface eth0 inet static
    address 192.0.2.7/24
    gateway 192.0.2.254
    dns-nameservers 1.1.1.1 8.8.8.8
```

DHCP Configuration Example

```
auto eth0
allow-hotplug eth0
iface eth0 inet dhcp
```

Note: In container-based environments (e.g., Docker or Kathara), the `ifup` command may not automatically update the `/etc/resolv.conf` file. DNS configuration may need to be adjusted manually.

3.1.2 Manual Network Configuration (Linux – Legacy)

In older (legacy) Linux systems, manual network configuration can be performed using command-line tools:

- `ifconfig` to assign the IP address:
  - Used to configure network interfaces or display their current configuration.
  - It can activate or deactivate interfaces using the `up` and `down` options.
  - General syntax:

    ```
    ifconfig <interface> <ip-address> netmask <netmask> up
    ```

- `route` to define the default gateway:
  - Used to display or modify the system routing table.
  - It allows the configuration of the default route toward a gateway.
  - General syntax:

    ```
    route add default gw <gateway-ip>
    ```

- `/etc/resolv.conf` to specify DNS server(s):
  - The DNS servers are defined by inserting lines such as:

    ```
    nameserver 8.8.8.8
    ```

3.1.3 Manual Network Configuration Using `ip` (Preferred)

Modern Linux systems use the `ip` command (from the `iproute2` suite) for network configuration.

- `ip addr` to assign IP addresses:
  - Used to configure network interfaces or display their current configuration.
  - Interfaces can be activated or deactivated using `ip link set`.
- `ip route` to define the default gateway:
  - Used to display or modify the routing table.
- `/etc/resolv.conf` to specify DNS server(s):
  - Add entries such as:

    ```
    nameserver 8.8.8.8
    ```

3.1.4 Other Details About the `ip` Command

- Show interfaces:

  ```
  ip link show
  ```

- Bring interface up/down:

  ```
  ip link set eth0 up
  ip link set eth0 down
  ```

- Set MAC address:

  ```
  ip link set eth0 address 00:11:22:33:44:55
  ```

- Show IP addresses:

  ```
  ip address show
  ip address show dev eth0
  ```

- Add/remove IP address:

  ```
  ip address add 10.0.0.1/8 dev eth0
  ip address del 10.0.0.1/8 dev eth0
  ```

- Flush all IP addresses:

  ```
  ip address flush dev eth0
  ```

3.1.5 `ip` for Routing Purposes

- List routing table:

```
ip route list
```

- Flush routing table:

```
ip route flush table main
```

- Add/delete routes:
  - Next hop:

```
ip route add 10.0.0.0/8 via 10.0.0.1
ip route del 10.0.0.0/8 via 10.0.0.1
```

  - Default route:

```
ip route add default via 10.0.0.1
ip route del default via 10.0.0.1
```

  - Direct forwarding:

```
ip route add 10.0.0.0/24 dev eth0
ip route del 10.0.0.0/24 dev eth0
```

3.1.6 `ip` for ARP and Advanced Features

- Show ARP cache:

```
ip neigh show
ip neigh show dev eth0
```

- Flush ARP cache:

```
ip neigh flush dev eth0
```

- Add/delete/change ARP entry:

```
ip neigh add 10.0.0.2 lladdr 00:11:22:33:44:55 dev eth0 nud permanent
ip neigh del 10.0.0.2 dev eth0
```

- IP tunneling (IP-in-IP, GRE, IPv6 tunneling):

```
ip tunnel
```

3.1.7 Utility Script: `subnet-info.sh`

The following Bash script can be used to compute subnet information starting from an IP address in CIDR notation:

subnet-info.sh

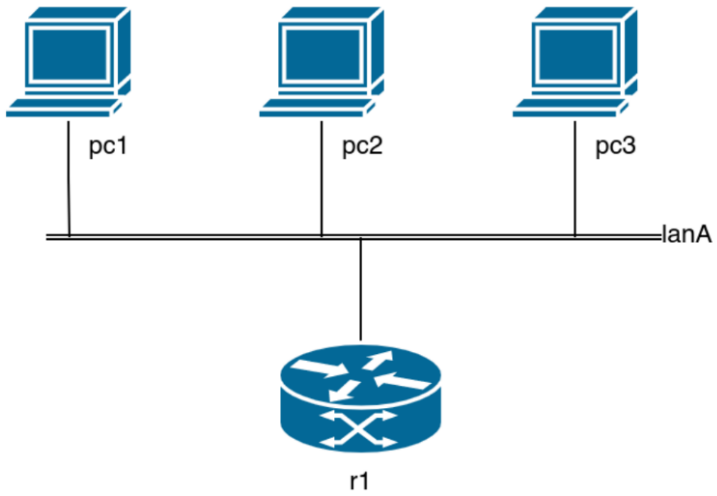
```
#!/bin/bash

if [ -z "$1" ]; then
  echo "Uso: ./subnet-info.sh IP/prefix"
  exit 1
fi

IP_CIDR=$1
IP=${IP_CIDR%/*}
PREFIX=${IP_CIDR#*/}

IFS=. read -r o1 o2 o3 o4 <<< "$IP"

IP_INT=$(( (o1<<24) + (o2<<16) + (o3<<8) + o4 ))

MASK=$(( 0xFFFFFFFF << (32 - PREFIX) & 0xFFFFFFFF ))

NETWORK=$(( IP_INT & MASK ))
BROADCAST=$(( NETWORK | ~MASK & 0xFFFFFFFF ))

HOST_BITS=$((32 - PREFIX))

if [ "$PREFIX" -ge 31 ]; then
  USABLE="Caso speciale (/31 o /32)"
  RANGE="N/A"
else
  USABLE=$((2**HOST_BITS - 2))
  FIRST=$((NETWORK + 1))
  LAST=$((BROADCAST - 1))
  RANGE="$(printf "%d.%d.%d.%d" $((FIRST>>24&255)) \
    $((FIRST>>16&255)) $((FIRST>>8&255)) $((FIRST&255))) \
    to $(printf "%d.%d.%d.%d" $((LAST>>24&255)) \
    $((LAST>>16&255)) $((LAST>>8&255)) $((LAST&255)))"
```

```
fi

to_ip() {
  printf "%d.%d.%d.%d" $(($1>>24&255)) \
    $(($1>>16&255)) $(($1>>8&255)) $(($1&255))
}

to_bin_octet() {
  local x=$1
  local b=""
  for ((i=7; i>=0; i--)); do
    if (( (x>>i)&1 )); then b+="1"; else b+="0"; fi
  done
  echo -n "$b"
}

to_bin() {
  local n=$1
  local a=$(( (n>>24)&255 ))
  local b=$(( (n>>16)&255 ))
  local c=$(( (n>>8)&255 ))
  local d=$(( n&255 ))
  printf "%s.%s.%s.%s" \
    "$(to_bin_octet $a)" \
    "$(to_bin_octet $b)" \
    "$(to_bin_octet $c)" \
    "$(to_bin_octet $d)"
}

echo ""
echo "1) IP address: $IP"
echo "2) Subnet mask: $(to_ip $MASK) (/$PREFIX)"
echo "3) IP binary representation:   $(to_bin $IP_INT)"
echo "4) Mask binary representation: $(to_bin $MASK)"
echo "5) Bitwise AND operation:      $(to_bin $NETWORK)"
echo "6) Network address: $(to_ip $NETWORK)"
echo "7) Broadcast address: $(to_ip $BROADCAST)"
```

```
echo "8) Number of host bits: 32 - $PREFIX = $HOST_BITS"

if [ "$PREFIX" -ge 31 ]; then
  if [ "$PREFIX" -eq 31 ]; then
    echo "9) Number of usable hosts: caso speciale /31 \
(tipicamente 2 per point-to-point)"
  else
    echo "9) Number of usable hosts: caso speciale /32 (1 indirizzo)"
  fi
  echo "10) Valid host range: $RANGE"
else
  TOTAL=$((2**HOST_BITS))
  USABLE=$((TOTAL - 2))
  echo "9) Number of usable hosts: 2^$HOST_BITS - 2 = \
$TOTAL - 2 = $USABLE"
  echo "10) Valid host range: $RANGE"
fi

echo ""
```

3.1.8 Exercise 1: `pnd-labs/lab1/ex1`

Consider the network topology shown below, composed of three hosts (`pc1`, `pc2`, `pc3`) connected to a router `r1` through the same LAN (lanA).



Objective:
Manually configure `pc1`, `pc2`, and `pc3` so that they belong to the same network as host `r1`, whose IP address is:

192.168.100.30/29

Requirements:

- Configure:
  - `pc1` using the `/etc/network/interfaces` file.
  - `pc2` using the `ip` command.
  - `pc3` using the `ifconfig` command.
- The DNS server can be the same DNS server used by the host machine.
  - This DNS configuration must also be specified in the `r1.startup` file.
- The default gateway for all PCs must be the `r1` host (already configured).
- Verify:
  - Connectivity within the local network.
  - Connectivity with the Internet (e.g., using `wget www.google.com`).

PC1 Configuration

We start by configuring `pc1`. Since `pc1` must belong to the same subnet as `r1` (which is already configured with `192.168.100.30/29`), we first compute the subnet parameters. We run the utility script to obtain network address, broadcast address, and valid host range:

```
user@kathara24:~/Desktop$ ./subnet-info.sh 192.168.100.30/29

1) IP address: 192.168.100.30
2) Subnet mask: 255.255.255.248 (/29)
3) IP binary representation:   11000000.10101000.01100100.00011110
4) Mask binary representation: 11111111.11111111.11111111.11111000
5) Bitwise AND operation:      11000000.10101000.01100100.00011000
6) Network address: 192.168.100.24
7) Broadcast address: 192.168.100.31
8) Number of host bits: 32 - 29 = 3
9) Number of usable hosts: 2^3 - 2 = 8 - 2 = 6
10) Valid host range: 192.168.100.25 to 192.168.100.30
```

From the output we derive that:

- Network: `192.168.100.24/29`
- Broadcast: `192.168.100.31`
- Usable host range: `192.168.100.25` – `192.168.100.30`

To configure `pc1` using the legacy `interfaces` mechanism, we create the required path `/etc/network/` inside the `pc1` directory and then create the file:

`/etc/network/interfaces`

We choose an IP address within the valid host range (e.g., `192.168.100.25/29`). The default gateway must be the router `r1`. In this topology, `r1` is configured with `192.168.100.30`, therefore:

$$\text{default gateway} = 192.168.100.30$$

Finally, we set a DNS server (in this example `8.8.8.8`).

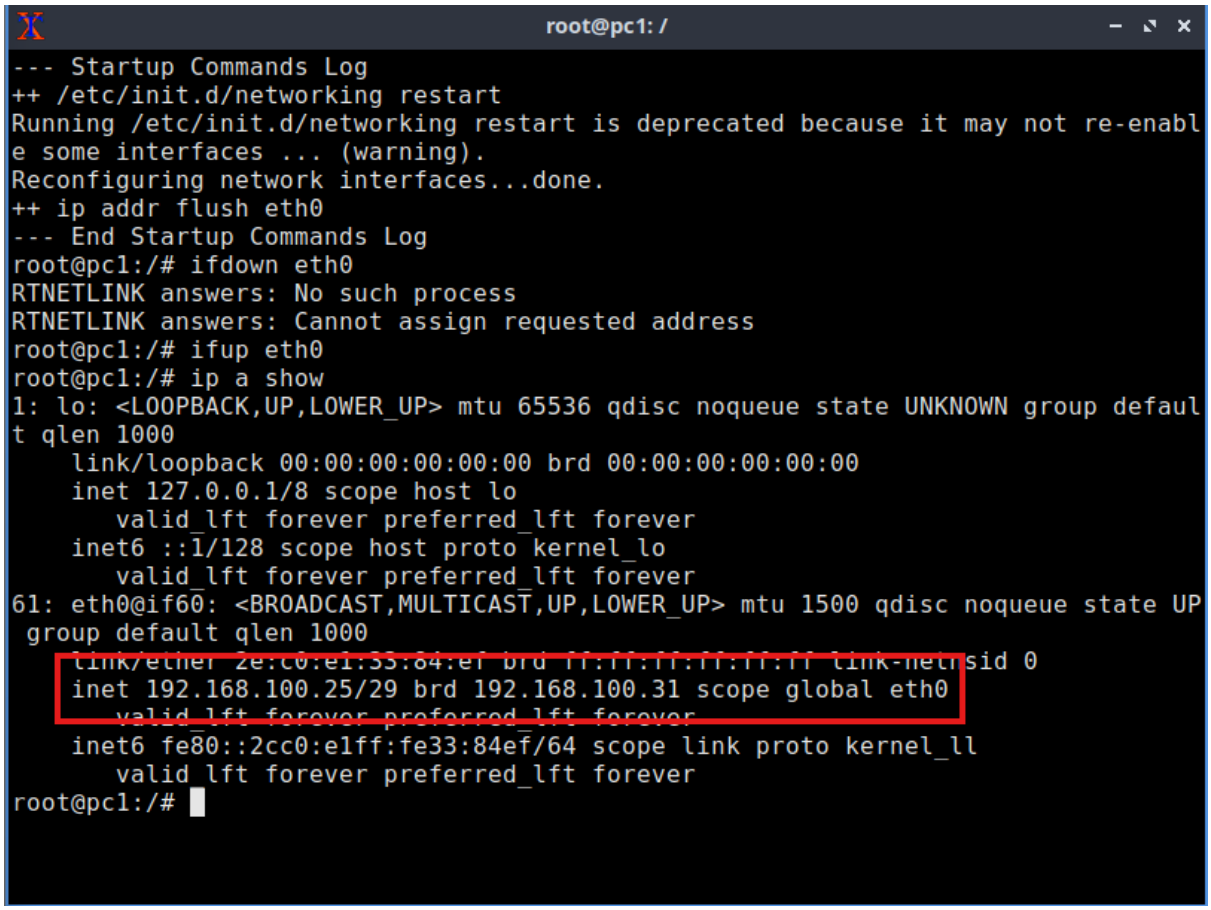
/etc/network/interfaces (pc1)

```
auto eth0
iface eth0 inet static
    address 192.168.100.25/29
    gateway 192.168.100.30
    dns-nameservers 8.8.8.8
```

At this point, `pc1` has a static IP configuration consistent with the subnet of `r1`. Remember to apply changes using:

```
ifdown eth0
ifup eth0
```

in pc1 terminal after you start the lab. If you write `ip a show` inside pc2 terminal, you should now see:



To ensure that the network configuration is applied automatically when the lab starts, the interface commands can be alternatively added to the `pc1.startup` file. This guarantees that the interface is properly reset and brought up every time the topology is initialized.

`pc1.startup`

```
ip addr flush eth0
ifdown eth0
ifup eth0
```

In this way, the interface configuration defined in `/etc/network/interfaces` is automatically applied at startup, avoiding manual intervention each time the lab is executed. It is important to notice that in this lab environment the command `ifup` does **not automatically update** the file `/etc/resolv.conf`. Therefore, even if the directive

`dns-nameservers 8.8.8.8`

is present inside `/etc/network/interfaces`, the DNS configuration may not be applied correctly.
For this reason, we must manually create the file:

`/etc/resolv.conf`

inside the `pc1` directory and explicitly specify the DNS server.

`/etc/resolv.conf (pc1)`

```
nameserver 8.8.8.8
```

Without this file, commands such as:

`wget www.google.com`

would fail with the error:

`Temporary failure in name resolution`

even if the routing and NAT configuration are correct.

PC2 Configuration

We now configure `pc2`. We assign the IP address `192.168.100.26`, which belongs to the valid host range of the subnet `192.168.100.24/29`.
From this point onward, instead of configuring the interface manually each time, we write the commands directly inside the `.startup` files so that the configuration is automatically applied when the lab starts.
First, we create the file:

`pc2/etc/resolv.conf`

and specify the DNS server:

`pc2/etc/resolv.conf`

```
nameserver 8.8.8.8
```

Then, inside the `pc2.startup` file, we configure the interface using `ifconfig` and define the default gateway using `route`. The default gateway must be the router `r1`, which has IP `192.168.100.30`.

pc2.startup

```
ip addr flush eth0
ifconfig eth0 192.168.100.26 netmask 255.255.255.248 up
route add default gw 192.168.100.30
```

The configuration will be automatically applied each time the lab is executed. If you write `ip a show` inside pc2 terminal, you should now see the IP address that we applied.

PC3 Configuration

We now configure `pc3` using the modern `ip` command.
We assign the IP address `192.168.100.27/29`, which belongs to the valid host range of the subnet `192.168.100.24/29`.
As done for `pc2`, the configuration commands are written directly inside the `pc3.startup` file so that they are automatically executed when the lab starts.
The DNS configuration for `pc3` is identical to `pc2`. We create the file:

`pc3/etc/resolv.conf`

with the following content:

pc3/etc/resolv.conf

```
nameserver 8.8.8.8
```

Then, inside the `pc3.startup` file, we configure the interface and define the default gateway using the `ip` command. The default gateway is the router `r1` with IP `192.168.100.30`.

pc3.startup

```
ip addr flush eth0
ip addr add 192.168.100.27/29 dev eth0
ip route add default via 192.168.100.30
```

The configuration is automatically applied at lab startup. If you write `ip a show` inside pc3 terminal, you should now see the IP address that we applied.

Connectivity Verification

After completing the configuration, it is possible to verify that everything is working correctly. First, we test the internal connectivity within the subnet by pinging between the PCs:

Internal connectivity test

```
ping 192.168.100.26   % from pc1 to pc2
ping 192.168.100.27   % from pc1 to pc3
```

If the configuration is correct, the hosts should reply, confirming that:

- the IP addresses are correctly assigned;
- the subnet mask is correct;
- the hosts belong to the same Layer 2 network.

Then, we verify Internet connectivity by pinging a public IP address:

Internet connectivity test

```
ping 8.8.8.8   % from each pc
```

If this works, it means that:

- the default gateway is correctly configured;
- routing is working properly.

Finally, we verify DNS resolution:

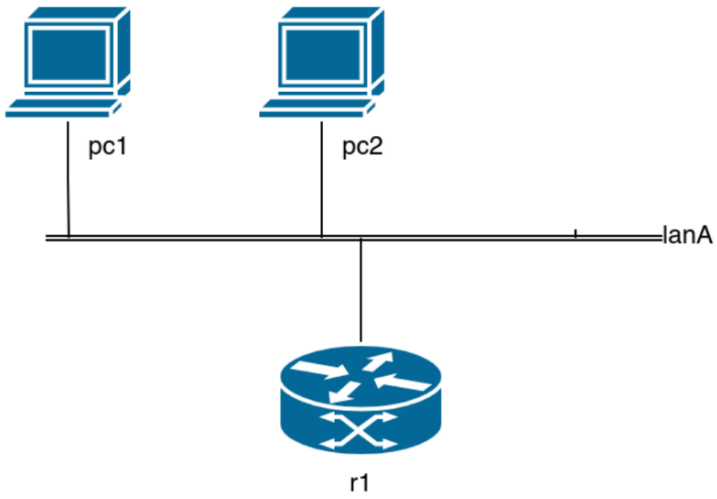
DNS resolution test

```
ping www.google.com   % from each pc
wget www.google.com   % from each pc
```

If this last test succeeds, then:

- the DNS server is correctly configured,
- the entire network configuration is consistent.

3.1.9 Exercise 2: `pnd-labs/lab1/ex2`



- Configure `r1`, `pc1` and `pc2` in order to receive their networking configuration from a DHCP server (`r1`)
  - The DNS server can be the server used by the host machine
  - The default gateway must be the `r1` machine, with IP address `192.168.100.30/28`
- Configure `r1` in order to operate as a DHCP server on the `eth0` interface
  - You can install `udhcpd` or any other server
    - (`apt install udhcpd`)
- Configure `pc1` using the `interfaces` file
- Configure `pc2` using the `dhclient` command (after run)
- Verify connectivity within the network and with the Internet (ex: `ping www.google.com`)

In this exercise, instead of manually assigning static IP addresses to all hosts, we configure `r1` as a DHCP server and allow clients to obtain their network configuration dynamically.

PC1 Configuration

We first configure `pc1` with a static IP address via `/etc/network/interfaces`:

pc1/etc/network/interfaces

```
auto eth0
iface eth0 inet static
    address 192.168.100.17/28
    gateway 192.168.100.30
    dns-nameservers 8.8.8.8
```

Since in this lab environment `ifup` does not automatically update `/etc/resolv.conf`, we manually create:

pc1/etc/resolv.conf

```
nameserver 8.8.8.8
```

To ensure automatic configuration at startup, we add the following commands to `pc1.startup`:

pc1.startup

```
ip addr flush eth0
umount /etc/resolv.conf
ifdown eth0
ifup eth0
```

The `flush` command removes any pre-existing IP configuration, while `umount` ensures that the local `resolv.conf` file is used instead of a mounted one.

R1 Configuration

The router `r1` is configured as a DHCP server using `udhcpd`.
The DHCP configuration file is:

r1/etc/udhcpd.conf

```
start 192.168.100.18
end   192.168.100.29

interface eth0
```

```
max_leases 12

opt dns 8.8.8.8
option subnet 255.255.255.240
opt router 192.168.100.30
option domain pnd.test
option lease 600
```

- The DHCP range excludes `192.168.100.17`, which is statically assigned to `pc1`.
- The router address `192.168.100.30` is excluded from the pool to avoid conflicts.
- The subnet mask corresponds to a /28 network (255.255.255.240).
- The lease time is set to 600 seconds (10 minutes).

The router startup configuration is:

r1.startup

```
ip addr replace 192.168.100.30/28 dev eth0
iptables -t nat -A POSTROUTING -o eth1 -j MASQUERADE

dpkg -i /var/cache/apt/archives/*.deb
apt install -f udhcpd

echo "nameserver 8.8.8.8" > /etc/resolv.conf

udhcpd -f /etc/udhcpd.conf
```

- The router IP is statically assigned.
- NAT (MASQUERADE) enables Internet connectivity.
- `udhcpd` is installed and started in foreground mode.

PC2 Configuration

Finally, `pc2` is configured as a DHCP client.
Its startup file contains:

pc2.startup

```
ip addr flush eth0
umount /etc/resolv.conf
dhclient eth0
```

The `dhclient` command performs the DHCP handshake:

- DHCP Discover
- DHCP Offer
- DHCP Request
- DHCP ACK

As a result, `pc2` automatically receives:

- An IP address within the configured pool
- The subnet mask
- The default gateway
- The DNS server

This completes the dynamic network configuration using DHCP.

Connectivity Verification

At this point, both internal and external connectivity should be correctly configured. Internal connectivity within the LAN can be verified by pinging between hosts:

Internal connectivity test

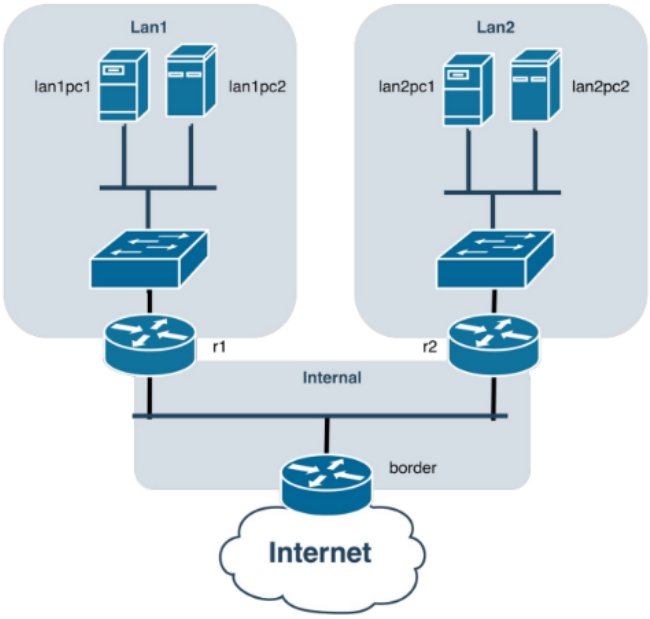
```
# From pc1
ping 192.168.100.X   # IP assigned to pc2

# From pc2
ping 192.168.100.17  # pc1 static IP
```

External connectivity can be verified by testing Internet access:

External connectivity test

```
ping 8.8.8.8
wget www.google.com
```

3.1.10 Exercise 3: `pnd-labs/lab1/ex3`



The network topology consists of:

- Two LANs: `lan1` and `lan2`
- Four PCs (two for each LAN)
- Three routers: `r1`, `r2`, and the border router `r0`

The router `r0` is already configured to act as the border gateway and is connected to the Internet through interface `eth1`. This interface must **not** be modified.

The objective of this exercise is to configure:

- The four PCs
- The three routers

so that:

- The two LANs are mutually reachable
- All hosts can reach the Internet
- The address space `172.16.0.0/16` must be used.
- Subnets must be assigned to all LANs and internal links in the topology. A suitable subnetting strategy must be chosen.

- The router `r0` must act as the default gateway for the entire network. It is already configured for this role and is connected to the Internet via `eth1`.
- Routers `r1` and `r2` must act as default gateways for `lan1` and `lan2`, respectively.
- `r1` and `r2` must:
  - Have a default route towards `r0`
  - Configure static routes to reach the opposite LAN
  - The `ip route` command can be used for this purpose
- The DNS server can be the one used by the host machine. It must be configured in all PCs.
- The PCs can be configured using any preferred method (static configuration, startup scripts, etc.).

Addressing Strategy

In this exercise the entire topology must use addresses from the private address space:

`172.16.0.0/16`

To simplify the network design, the address space is divided into three subnets (see addr-assignment.txt):

- **LAN1**: `172.16.1.0/24`
- **LAN2**: `172.16.2.0/24`
- **Internal router network**: `172.16.254.0/24`

The two LANs host the PCs, while the internal network is used exclusively for communication between routers.

- Router `r1` connects `lan1` to the internal router network.
- Router `r2` connects `lan2` to the internal router network.
- Router `r0` acts as the **border router** and provides Internet connectivity.

Each LAN uses a local DHCP server running on its router (`r1` for LAN1 and `r2` for LAN2), so that hosts automatically receive their configuration.

The main addresses used in the topology are:

- **r1**
  - `172.16.1.1/24` on eth1 (LAN1)
  - `172.16.254.1/24` on eth0 (internal router network)
- **r2**
  - `172.16.2.1/24` on eth1 (LAN2)
  - `172.16.254.254/24` on eth0 (internal router network)
- **r0**
  - `172.16.254.128/24` on eth0 (internal router network)

Router `r1` Configuration

Router `r1` connects LAN1 to the internal router network and acts as the default gateway for the hosts in LAN1.
Its startup configuration is:

`r1.startup`

```
ip addr add 172.16.1.1/24 dev eth1
ip addr add 172.16.254.1/24 dev eth0
ip route add 172.16.2.0/24 via 172.16.254.254
ip route add default via 172.16.254.128

dpkg -i /var/cache/apt/archives/*.deb
apt install -f udhcpd

echo "nameserver 8.8.8.8" > /etc/resolv.conf

udhcpd -f /etc/udhcpd.conf
```

This configuration performs the following operations:

- Assigns the IP address `172.16.1.1/24` to interface `eth1`, which connects to LAN1.
- Assigns the IP address `172.16.254.1/24` to interface `eth0`, which connects to the internal router network.
- Adds a static route to reach LAN2 (`172.16.2.0/24`) through router `r2` (`172.16.254.254`).
- Configures a default route towards the border router `r0` (`172.16.254.128`), which provides Internet access.

- Installs and starts the DHCP server `udhcpd`.

The DHCP configuration for LAN1 is:

r1/etc/udhcpd.conf

```
start 172.16.1.100
end   172.16.1.200

interface eth1
max_leases 100

opt dns 8.8.8.8
option subnet 255.255.255.0
opt router 172.16.1.1
option domain pnd.test
option lease 600
```

This DHCP server dynamically assigns IP addresses to hosts in LAN1 within the range:

$$172.16.1.100 \; - \; 172.16.1.200$$

and provides:

- subnet mask
- default gateway (`172.16.1.1`)
- DNS server (`8.8.8.8`)

The range of IP addresses is my design choice, you can assign within the range:

$$172.16.1.2 \; - \; 172.16.1.254$$

Router `r2` Configuration

Router `r2` connects LAN2 to the internal router network and acts as the gateway for hosts in LAN2.

r2.startup

```
ip addr add 172.16.2.1/24 dev eth1
ip addr add 172.16.254.254/24 dev eth0
ip route add 172.16.1.0/24 via 172.16.254.1
ip route add default via 172.16.254.128
```

```
dpkg -i /var/cache/apt/archives/*.deb
apt install -f udhcpd

echo "nameserver 8.8.8.8" > /etc/resolv.conf

udhcpd -f /etc/udhcpd.conf
```

This configuration:

- Assigns `172.16.2.1/24` to the LAN2 interface.
- Assigns `172.16.254.254/24` to the internal router network.
- Adds a static route to LAN1 through router `r1`.
- Configures the default route towards the border router `r0`.

The DHCP configuration for LAN2 is:

r2/etc/udhcpd.conf

```
start 172.16.2.100
end   172.16.2.200

interface eth1
max_leases 100

opt dns 8.8.8.8
option subnet 255.255.255.0
opt router 172.16.2.1
option domain pnd.test
option lease 600
```

Hosts in LAN2 therefore automatically receive addresses from the range:

$$172.16.2.100 \; - \; 172.16.2.200$$

The range of IP addresses is my design choice, you can assign within the range:

$$172.16.2.2 \; - \; 172.16.2.254$$

Border Router `r0` Configuration

Router `r0` acts as the **border gateway** for the entire topology and provides Internet connectivity.

`r0.startup`

```
ip addr add 172.16.254.128/24 dev eth0
ip route add 172.16.1.0/24 via 172.16.254.1
ip route add 172.16.2.0/24 via 172.16.254.254
iptables -t nat -A POSTROUTING -o eth1 -j MASQUERADE

echo "nameserver 8.8.8.8" > /etc/resolv.conf
```

This router performs two key functions:

- It forwards traffic between the internal routers and the Internet.
- It performs **NAT (Network Address Translation)** using the `MASQUERADE` rule.

Static routes are added so that `r0` knows how to reach:

- LAN1 via router `r1`
- LAN2 via router `r2`

PCs Configuration

All PCs in both LANs are configured as DHCP clients. Their startup configuration is identical:

`lanXpcY.startup`

```
ip addr flush eth0
umount /etc/resolv.conf

dhclient eth0
```

The command sequence performs the following steps:

- Removes any pre-existing IP configuration from the interface.
- Ensures that the local `/etc/resolv.conf` file is used.
- Starts the DHCP client (`dhclient`) to obtain network configuration automatically.

Each PC also contains the file:

lanXpcY/etc/resolv.conf

```
nameserver 8.8.8.8
```

Once the DHCP negotiation is completed, each host receives:

- an IP address from the DHCP pool
- the subnet mask
- the default gateway (either `r1` or `r2`)
- the DNS server

Connectivity Verification

After the configuration is completed, connectivity can be verified at three different levels.

1. LAN connectivity

Hosts within the same LAN should be able to reach each other:

LAN connectivity test

```
ping <ip-other-host-in-same-lan>
```

2. Inter-LAN connectivity

Hosts in LAN1 should be able to reach hosts in LAN2 and vice versa. This verifies that the static routes on `r1` and `r2` are correctly configured.

Inter-LAN connectivity test

```
ping 172.16.2.X   # from LAN1
ping 172.16.1.X   # from LAN2
```

3. Internet connectivity

Finally, Internet access can be tested through the border router:

Internet connectivity test

```
ping 8.8.8.8
ping www.google.com
wget www.google.com
```

If these tests succeed, the entire routing infrastructure (`r1`, `r2`, and `r0`), the DHCP configuration, and the NAT mechanism are functioning correctly.

3.2 Laboratory of 05/03/26

DISCLAIMER

Some images in this section may appear small or difficult to read at the default size. It may be necessary to **zoom into the figures** in order to clearly see all the details.

3.2.1 Installing Wireshark and Permissions

If you try to install Wireshark on your machine, there is one important step that should be performed at the very beginning.
In order to put the network interface into **promiscuous mode**, special permissions are required, typically **root privileges**. For this reason, one might be tempted to run Wireshark using `sudo`.
However, this is not a safe approach. Running Wireshark with root permissions is generally discouraged. Wireshark processes a large amount of network traffic, and some of this traffic may be **malicious**. In certain situations, attackers could attempt to exploit vulnerabilities in Wireshark itself.
If Wireshark were compromised while running with **root privileges**, the attacker would gain control of a **superuser process**, which would pose a serious security risk.
A safer solution is to allow the Wireshark user to capture packets without running the entire application as root. This approach follows the **principle of least privilege**, which states that a program should only be granted the permissions that are strictly necessary to perform its task.
In practice, this means that the user must be added to the appropriate system group that allows packet capturing. After doing so, it will be possible to run Wireshark normally, without superuser privileges, while still being able to capture network traffic.

We first install `tshark` and then the full Wireshark GUI package:

Installing Wireshark components

```
sudo apt install tshark
sudo apt install wireshark
```

During the installation of `wireshark-common`, the package configuration asks an important question:

> *Should non-superusers be able to capture packets?*

In this case, the correct choice is `Yes`. By enabling this option, the system configures `dumpcap` so that packet capture can be delegated to members of the dedicated `wireshark` system group, instead of forcing the whole Wireshark process to run as `root`.

Before changing group membership, we check the groups currently associated with the user:

Checking the current groups

```
groups
```

The output shows:

```
user adm cdrom sudo dip plugdev lpadmin sambashare
```

At this point, the user is **not yet** a member of the `wireshark` group.
To grant the required permission, the user is added to the group with:

Adding the user to the `wireshark` group

```
sudo usermod -aG wireshark user
```

The meaning of the options is the following:

- `-a` means *append*, so the user keeps all existing groups;
- `-G wireshark` specifies the supplementary group to add;
- `user` is the username that must be modified.

It is important to use `-aG` together. If the append option `-a` were omitted, the user could be removed from all previously assigned supplementary groups and kept only in the specified one.

The operation succeeds, however, group membership changes are not applied to the current shell session immediately. For this reason, it is necessary to log out and log back in (or reboot the machine). After starting a new session, we run again:

Verifying the new group membership

```
groups
```

This time the output becomes:

```
user adm cdrom sudo dip plugdev lpadmin wireshark sambashare
```

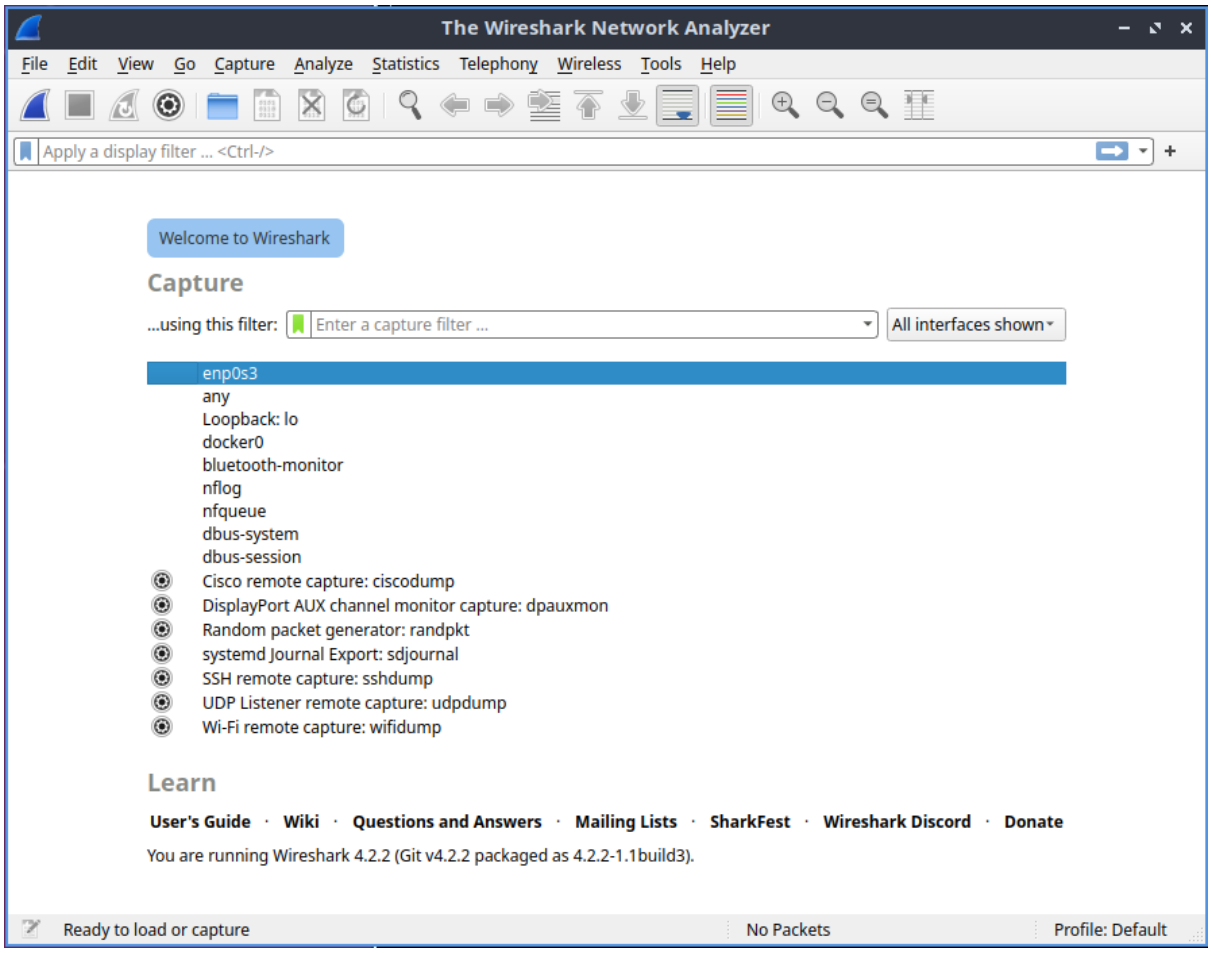
We can now see that the `wireshark` group is present, which confirms that the user has been correctly added.

Wireshark can now be launched directly from the terminal without using `sudo`. In other words, it is sufficient to run:

Starting Wireshark

```
wireshark
```

Once Wireshark is open, it is also possible to improve readability by changing the font and the font size used in the interface.



To customize the text appearance in Wireshark, open the menu:

`Edit` → `Preferences`

Inside the Preferences window, select:

`Appearance` → `Font and Colors`

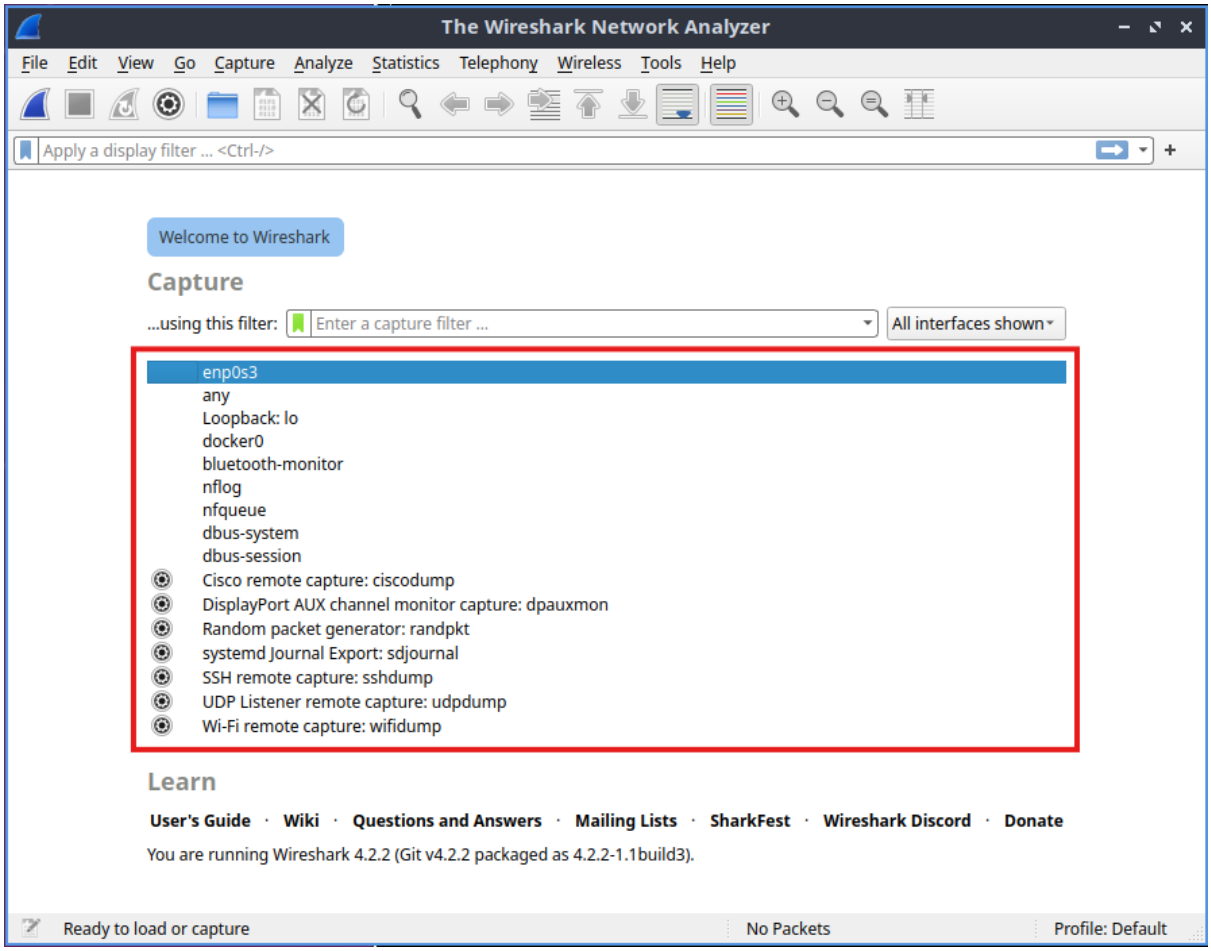
In this section, the field labeled `Main window font` shows the font currently used by Wireshark. By clicking on this field, a font selection window is opened.
From there, it is possible to choose:

- the **font family** (for example `Liberation Mono`);
- the **font style** (for example regular, italic, bold);

- the **font size**.

After selecting the desired options, press `OK` to confirm the font selection, and then press `OK` again in the Preferences window to apply the changes.

3.2.2 Wireshark Capture Interfaces



When opening Wireshark, several network interfaces are available for capturing traffic. One of these is a **pseudo-interface** called `any`. This interface aggregates traffic coming from different network interfaces and presents it as if it were a single interface. In this way, it is possible to capture packets from multiple networks simultaneously.
Another important interface is the **loopback interface**. The loopback interface refers to the **machine itself**. Its main purpose is to enable communication between processes running on the same host.
Typically, the loopback interface is used as a mechanism for **inter-process communication** within the same machine. For example, consider a scenario in which a **web server** needs to communicate with a **database**, and both services are running on the same host.

In this case, the communication still goes through the **network stack**, using protocols such as IP and TCP, together with the usual encapsulation mechanisms. However, the traffic does not actually leave the host machine.
This approach also provides an additional security benefit. If a service only accepts connections from `localhost`, then only processes running on the same machine can access that resource. As a result, the service is not exposed to external networks.
We can also observe an interface related to **Docker**, usually called `docker0`. Docker is a virtualization platform that allows the creation of **containerized environments**. Each container behaves like a lightweight virtual machine and communicates through virtual network interfaces. This interface enables communication between the host and the Docker containers. In our exercises, Docker will also be used to simulate different networked machines.
The list also includes several other technical interfaces such as `dbus-system`, `dbus-session`, `nflog`, `nfqueue`, and `bluetooth-monitor`. These are related to specific system services or kernel-level networking mechanisms. For the purpose of our analysis, we do not need to focus on them.
The most relevant interface for us is `enp0s3`. This is the **physical network interface** of the machine. The name follows a modern Linux **predictable naming convention** for network interfaces. The different components of the name reflect the hardware location of the network device in the system (for example the controller and the slot where the device is connected).
In systems with multiple network controllers and several physical ports, this naming scheme helps uniquely identify each interface. In this case, the interface is named `enp0s3`, which simply follows this convention.
For our purposes, this is the interface that actually connects the machine to the network, and therefore it is the one we will use to capture traffic.

3.2.3 Capturing on `enp0s3`

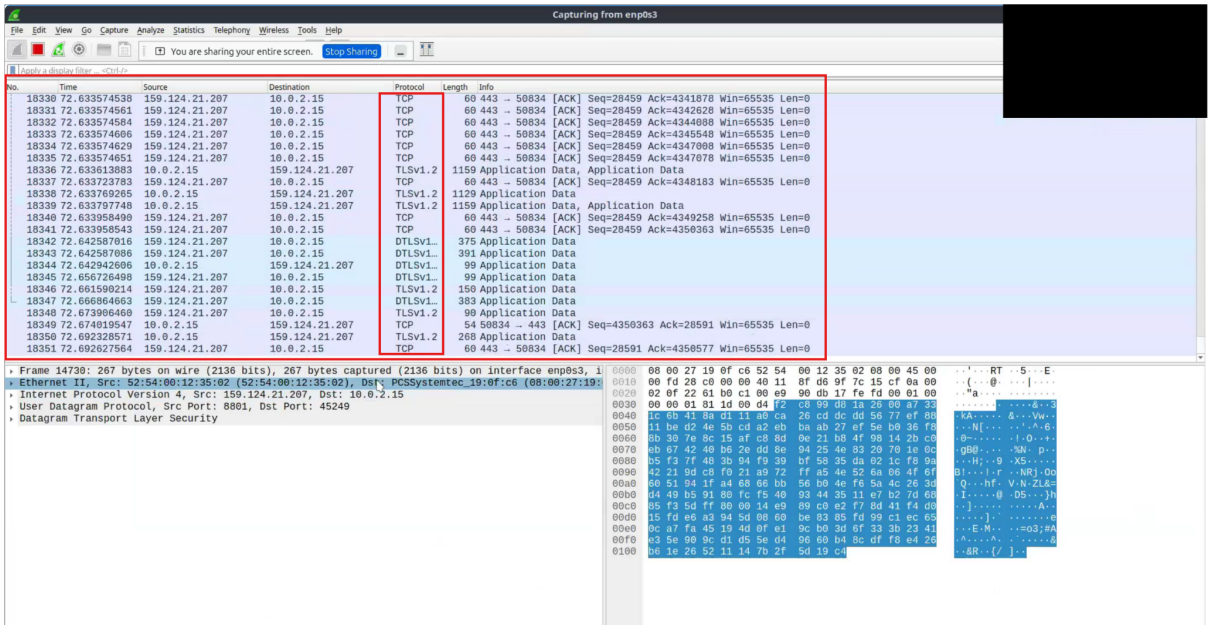


Figure 13: Professor's View

To begin capturing traffic in Wireshark, we simply double-click on the desired network interface. Once the capture starts, Wireshark begins to **sniff network traffic** and displays the packets in real time.

It is important to clarify that in this case we are **not capturing traffic from a physical Ethernet interface**. The system is running inside a **virtual machine**, therefore the interface `enp0s3` that we previously selected is actually a **virtual network interface**. This interface is created and managed by **VirtualBox**.

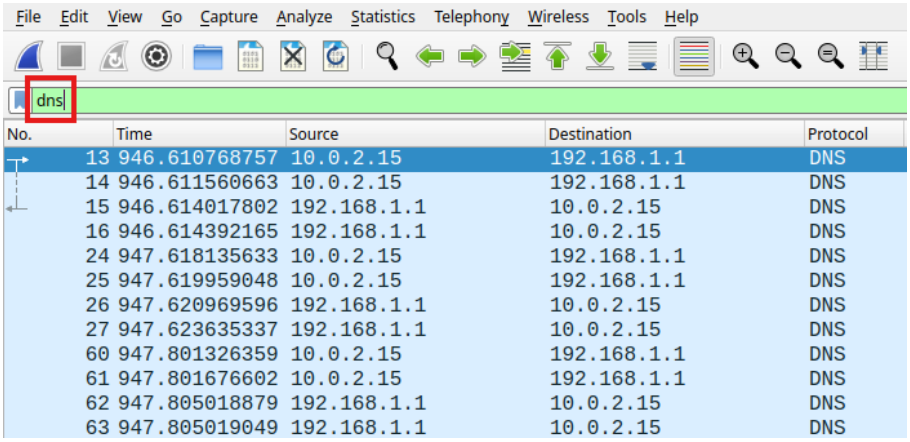
In practice, the virtual machine generates traffic through this virtual interface. VirtualBox then takes these packets and forwards them to the **physical network interface** of the host machine, which is the real interface connected to the network.

As soon as the capture starts, Wireshark immediately begins collecting a large amount of packets. It is normal to observe thousands of packets in a short period of time.

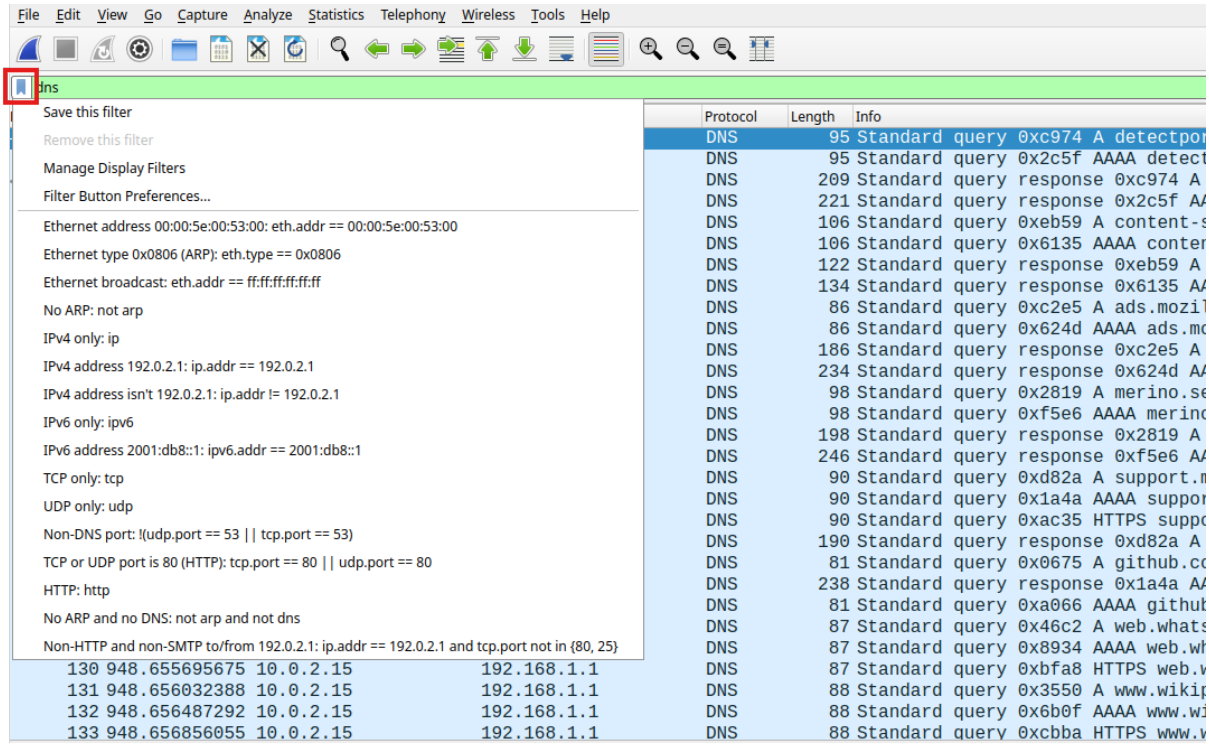
This happens because the system is continuously generating network traffic. For example, in the previously shown scenario the machine is connected to **Zoom** and is streaming the screen. Video conferencing applications constantly exchange data packets, sending and receiving hundreds of bytes every second. As a result, Wireshark quickly captures a large number of packets.

Capture Filters and Display Filters

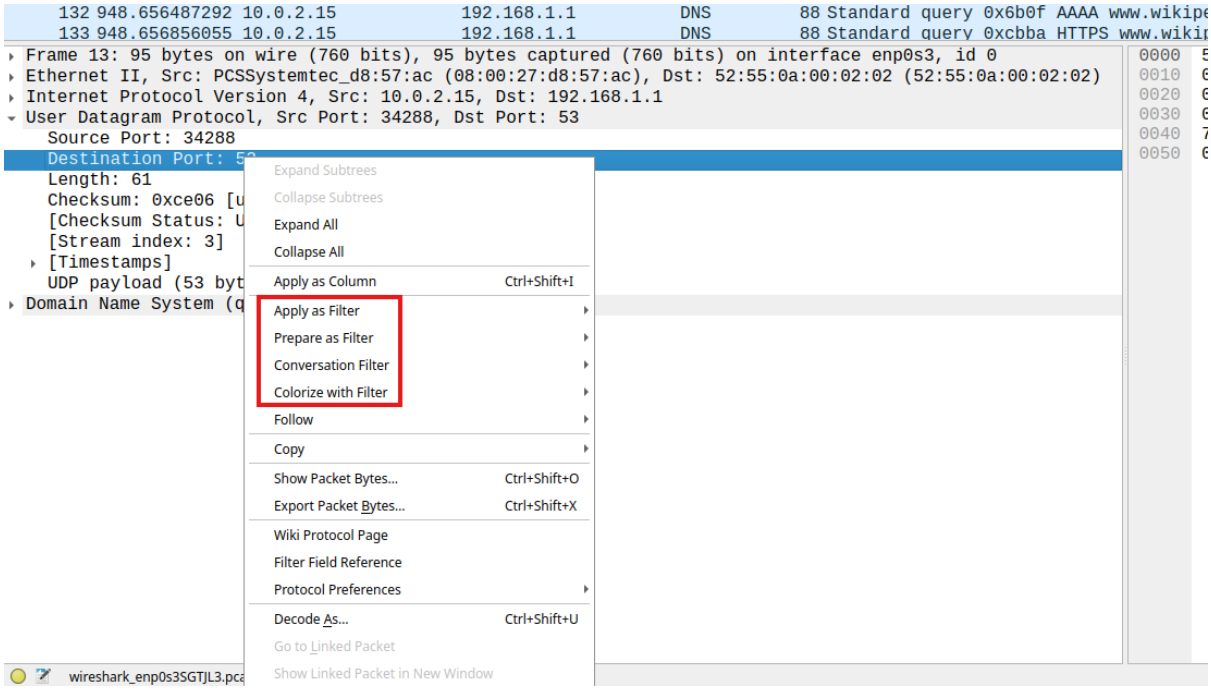
Wireshark may capture a large amount of traffic, but the user might only be interested in analyzing a specific type of packet. For this reason, Wireshark provides two types of filters: **Capture Filters** and **Display Filters**.



The **display filter** can be entered in the filter bar at the top of the Wireshark interface. For example, if we want to visualize only DNS traffic, we can type `dns`. After applying the filter, Wireshark will display only the packets related to the **DNS protocol**, hiding all the other captured packets. This allows us to focus on a specific type of network communication and analyze it more easily.



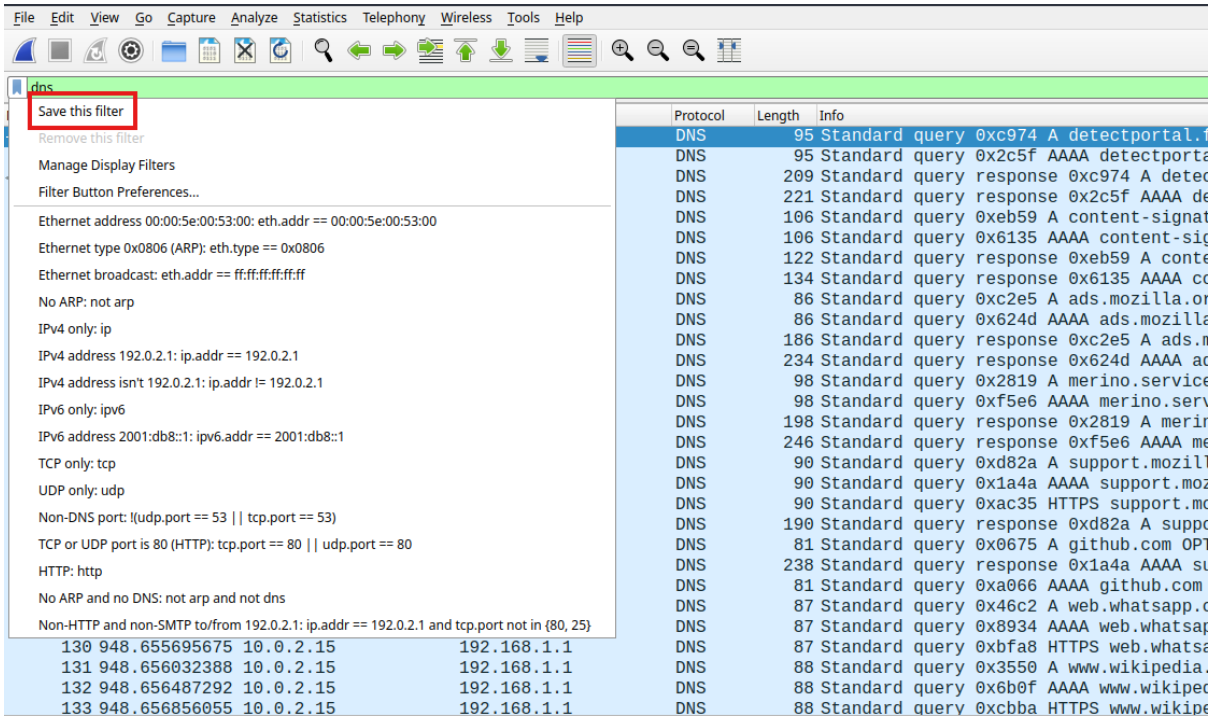
Wireshark also provides a convenient **filter builder** that helps users create filters using predefined templates. By clicking on the bookmark icon next to the filter bar, a menu appears containing several predefined filters, which can be selected and applied directly.



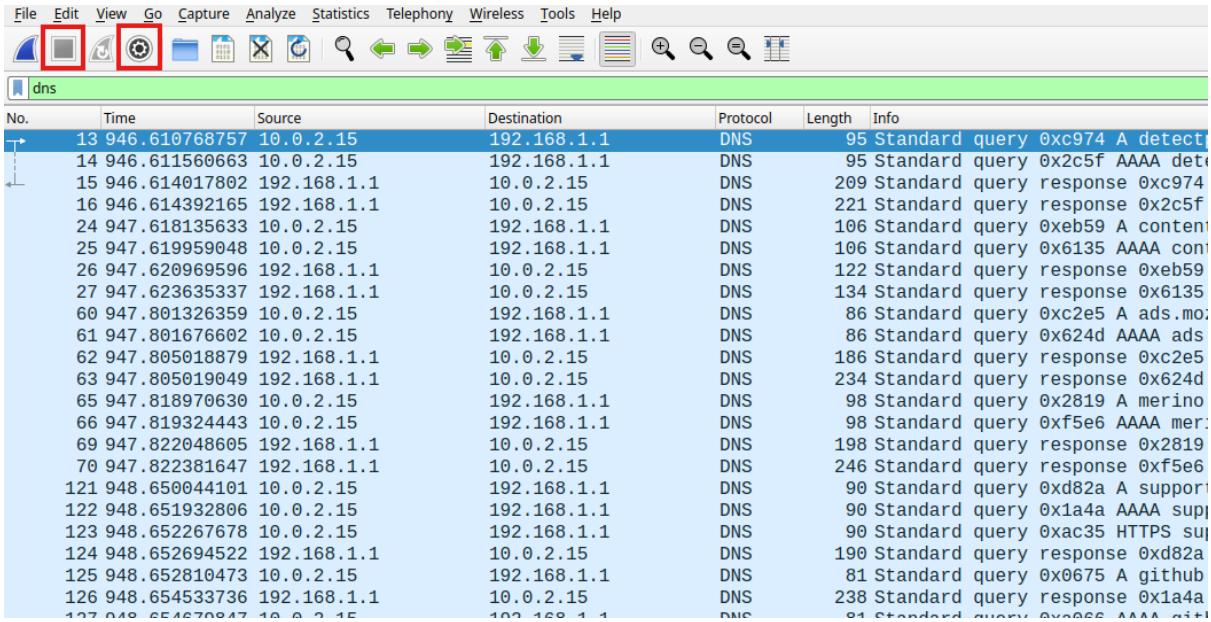
Another useful feature is the ability to build filters directly from packet fields. By selecting a packet and expanding its protocol fields in the packet details panel, it is possible to right-click on any field and create a filter based on that value. There are several options, including:

- **Apply as Filter**: immediately applies the filter to the current capture.
- **Prepare as Filter**: inserts the filter into the filter bar without applying it, allowing further modifications before execution.

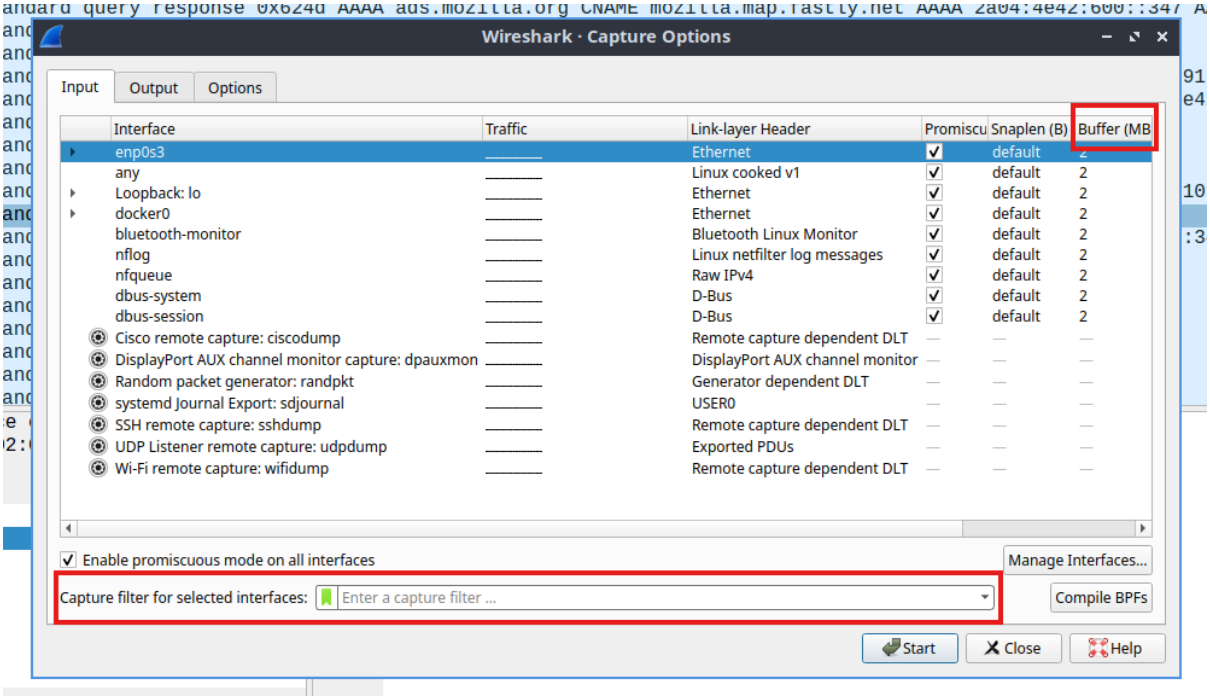
This mechanism makes it very easy to construct precise filters without manually writing the entire expression.



Once a display filter has been created, it can also be **saved for later use**. Saved filters appear in the list of available filters and can be quickly reused during future analyses.



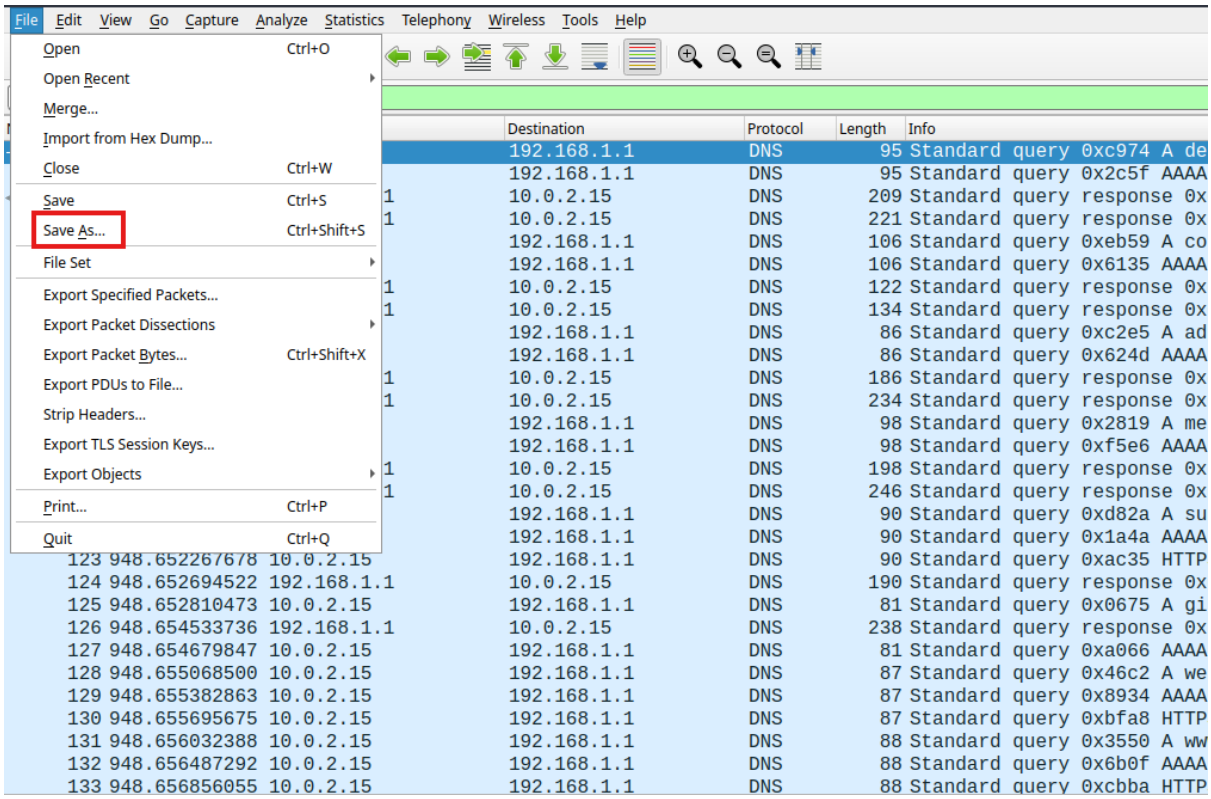
Capture filters can be configured before starting the packet capture (or you can stop the ongoing capture to configure them).



To access these options, we can open the **Capture Options** window, inside which it is possible to specify the filter in the field labeled `Capture filter for selected interfaces`. Capture filters are typically less expressive than display filters because they operate at a **lower level**. They are applied directly during the packet capture process and therefore must be efficient enough to run at high speed while packets are arriving.

When a network generates a large amount of traffic, capturing every packet may quickly consume a large amount of **memory**. For this reason, Wireshark uses a **capture buffer** (measured in MB) to temporarily store packets while they are being processed.
If the incoming traffic rate is very high (for example in the order of megabits per second), the interface may struggle to capture all packets. In such situations, capture filters become essential because they reduce the amount of traffic that must be stored and analyzed.
In practice, packet analysis is usually performed with a specific goal in mind. For example, if we know that a particular host is involved in the analysis, we can configure a capture filter that captures only the traffic related to that host.

Saving Captured Traffic



After capturing packets, Wireshark allows us to save the captured data for later analysis. This can be done using the **Save As** option.
Captured traffic is typically stored in the **PCAP** format, which is the standard file format used for packet captures. A newer version called **PCAPNG** (PCAP Next Generation) is also available and provides additional features.
Saving packet captures is very useful because the same capture file can later be:

- reopened and analyzed again;
- shared with other analysts;
- filtered and inspected multiple times;
- used for further network debugging or forensic analysis.

GeoIP Resolution

Another useful feature provided by Wireshark is the **GeoIP resolver**. When this option is enabled, Wireshark can associate IP addresses with their approximate **geographical location**.
This functionality relies on external **GeoIP databases**, which map IP address ranges to geographical information such as:

- country
- region
- city
- latitude and longitude

By using these databases, Wireshark can enrich captured packets with geographical information and make it possible to apply filters based on location.
For example, it becomes possible to create display filters that show only packets coming from specific geographical areas, such as:

- traffic originating from a specific **country** (e.g., China);
- traffic coming from a specific **region** (e.g., Lazio);
- traffic associated with a particular **city** (e.g., Rome).

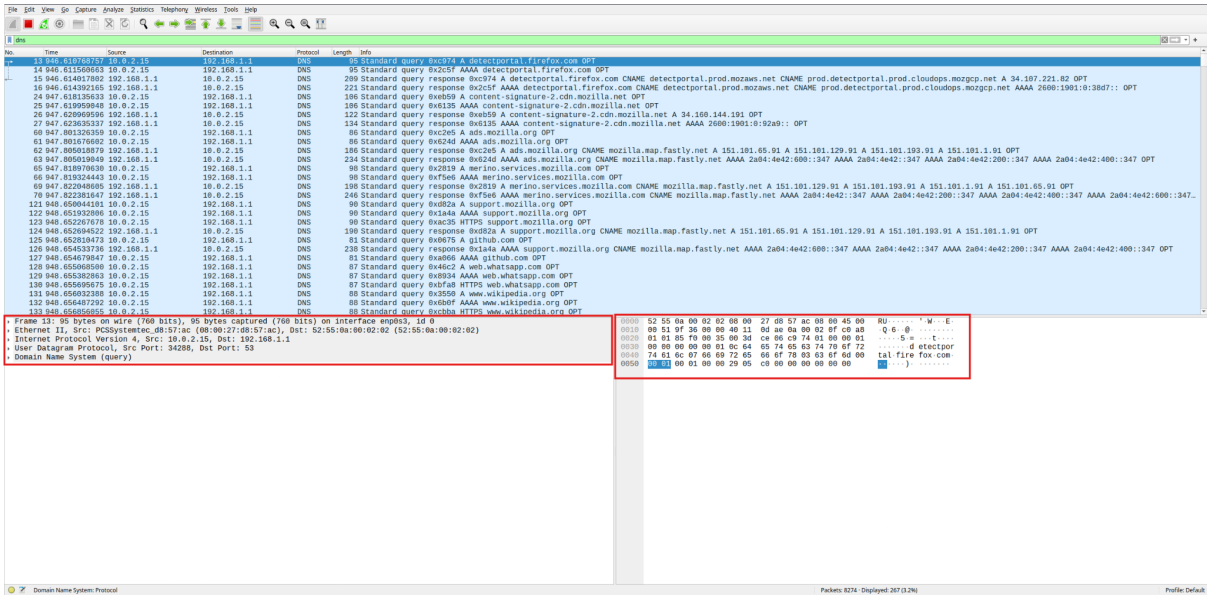
This feature can be useful for network analysis, security investigations, or identifying suspicious traffic coming from certain parts of the world.
The GeoIP functionality can be enabled by configuring Wireshark with the appropriate GeoIP database files, after which the geographical information becomes available for filtering and analysis (see the slides for the tutorial).

Wireshark Interface

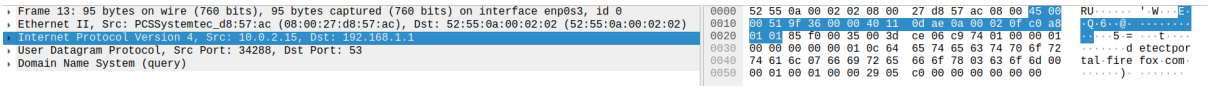
The Wireshark interface is divided into several sections:

- The **top panel** shows the list of captured packets.
- The **bottom left panel** shows the protocol dissection of the selected packet.
- The **bottom right panel** displays the raw packet data in hexadecimal format.

By clicking on a packet in the top panel, it is possible to inspect its internal structure and analyze the protocol layers involved in the communication.



After applying the display filter `dns`, Wireshark shows only the packets related to DNS traffic. In the current capture we can see that **8274 packets** have been captured in total, but only **267 packets** are currently displayed (about **3.2%**) because of the filter.
If we select a packet from the list (for example the first DNS query), Wireshark displays several additional details in the lower panels.



When clicking on one of the protocol layers in the packet details panel, Wireshark highlights the corresponding bytes in the hexadecimal view:

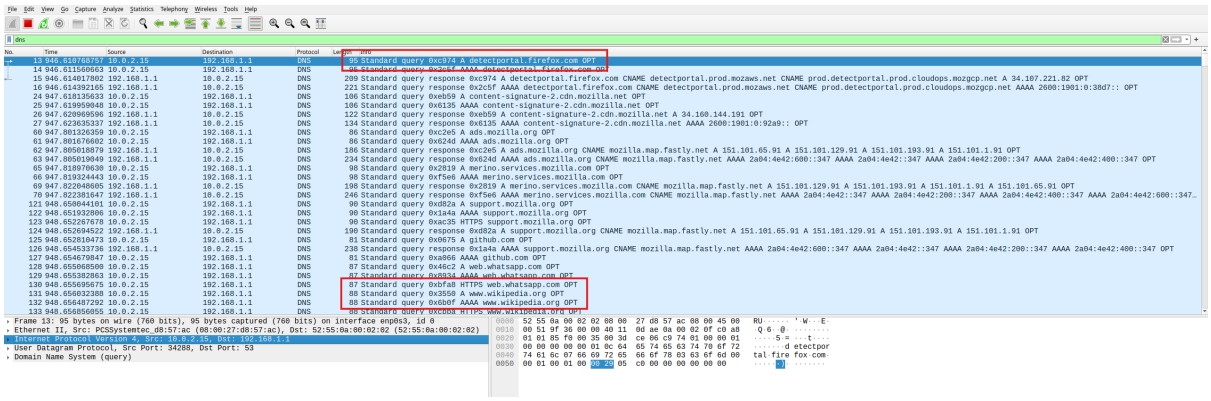
- selecting **Frame** highlights the entire packet;
- selecting **Ethernet II** highlights only the Ethernet header;
- selecting **IP** highlights the IP header;
- selecting **UDP** highlights the UDP header;
- selecting **DNS** highlights only the application-layer DNS data.

This feature helps to understand how the packet is structured and how the different protocol layers encapsulate each other.

Outgoing vs Incoming Packets

There is also a subtle difference between packets **generated by the machine** and packets **received from the network**.
Outgoing packets are captured by the system **before** they are placed on the physical medium. Because of this, some Ethernet-level elements (such as the preamble or the frame control sequence) are not yet added at the moment of capture.
On the other hand, packets that are received from the network may include additional information added at the link layer. For this reason, the number of bytes reported in the frame may sometimes appear larger than the meaningful payload displayed in the packet dissection.

Packet Summary Information



In the packet list, Wireshark provides a compact summary of each packet. For example, a row may show something like:

`Standard query A detectportal.firefox.com`

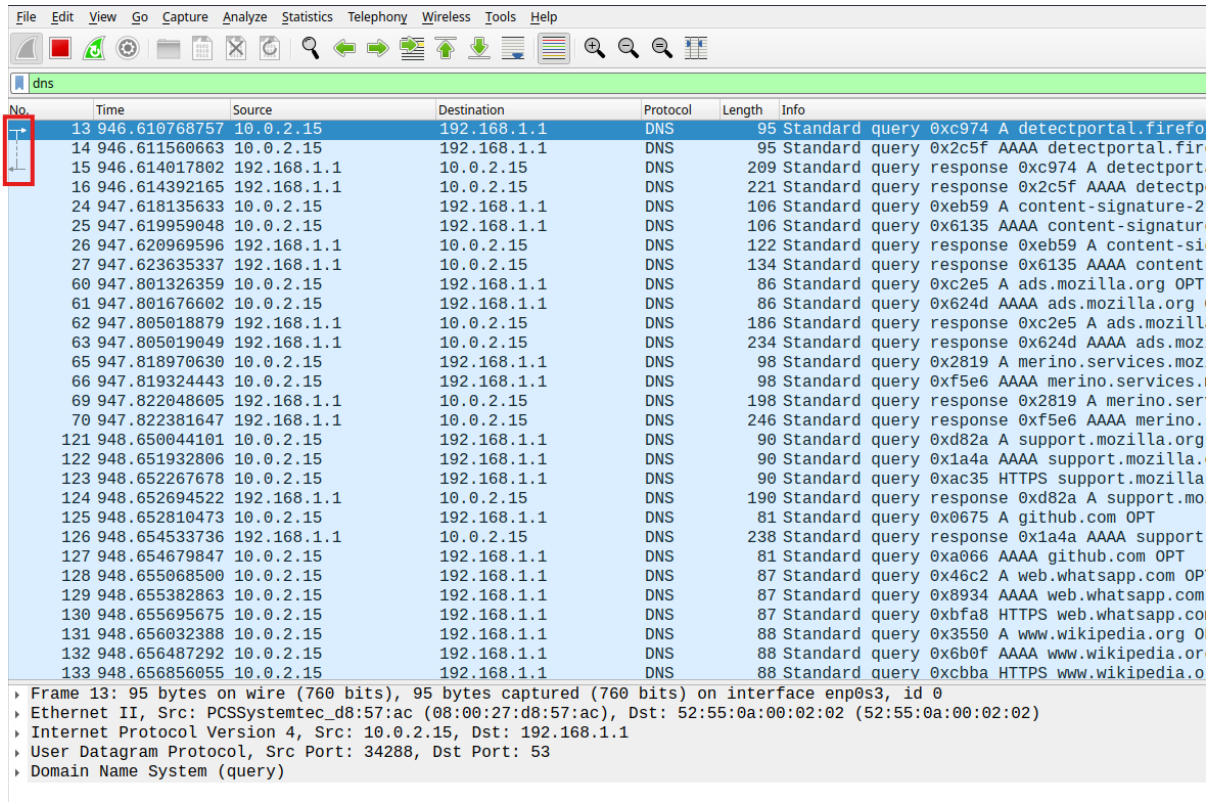
This summary indicates that the packet is a **DNS query** requesting the **IPv4 address** (record type `A`) for the domain `detectportal.firefox.com`.
Wireshark can infer the protocol type by inspecting the packet contents. In this case, it recognizes the DNS protocol because the UDP **destination port is 53**, which is the standard port used by DNS.
When a browser performs a DNS lookup, it often sends multiple queries:

- **A record**: requests the IPv4 address of a domain.
- **AAAA record**: requests the IPv6 address.
- sometimes additional queries (for example HTTPS-related records).

For example, in the capture we can see queries such as:

`Standard query A www.wikipedia.org`
`Standard query AAAA www.wikipedia.org`

This happens because modern systems try to obtain both IPv4 and IPv6 addresses.



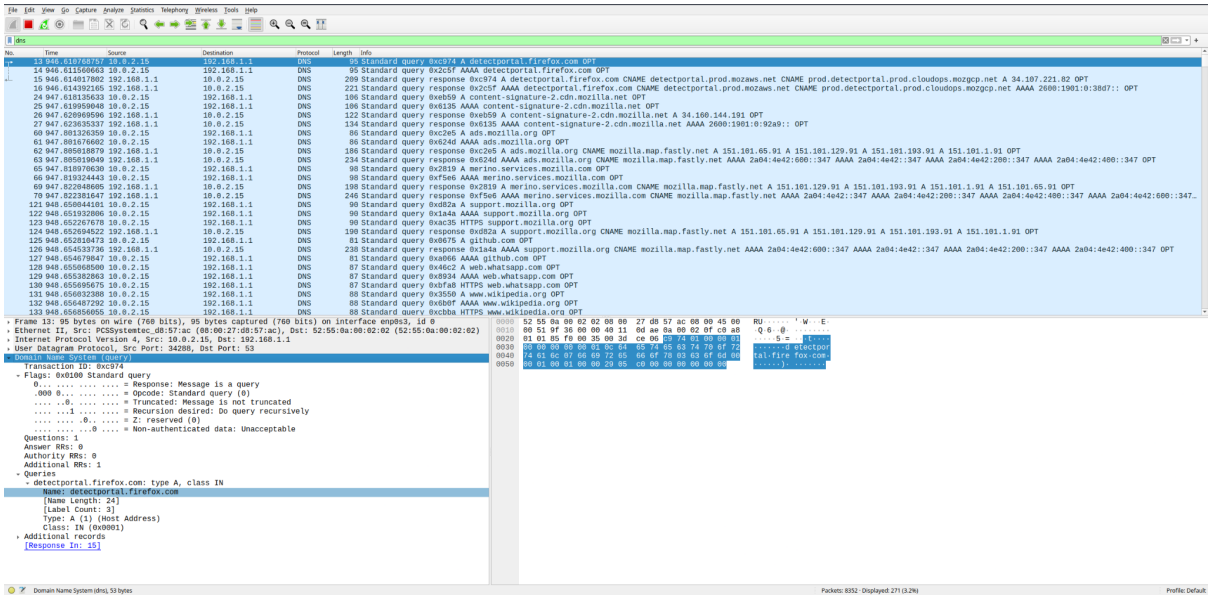
Wireshark also helps correlate **DNS requests and responses**. If we select a query packet, Wireshark shows a small arrow linking it to the corresponding response packet. For example:

- a packet labeled `Standard query` represents the DNS request;
- a packet labeled `Standard query response` represents the DNS reply from the DNS server.

This feature makes it easier to follow the interaction between a client and a DNS server and to understand how domain names are resolved into IP addresses.

Exploring Packet Details

When selecting a packet in Wireshark, it is possible to inspect the packet structure down to the level of individual bytes. For example, if we expand the **Domain Name System (DNS)** section in the packet details panel, Wireshark shows the internal structure of the DNS message and explains the meaning of each field according to the protocol format.



In the example shown in the capture, the packet corresponds to a **standard DNS query**. Wireshark displays several fields, including:

- **Transaction ID**: used to match DNS requests and responses.
- **Flags**: indicating properties of the message (e.g., whether it is a query or a response).
- **Questions**: the number of queries contained in the message.
- **Answer RRs**, **Authority RRs**, and **Additional RRs**: resource record counters.

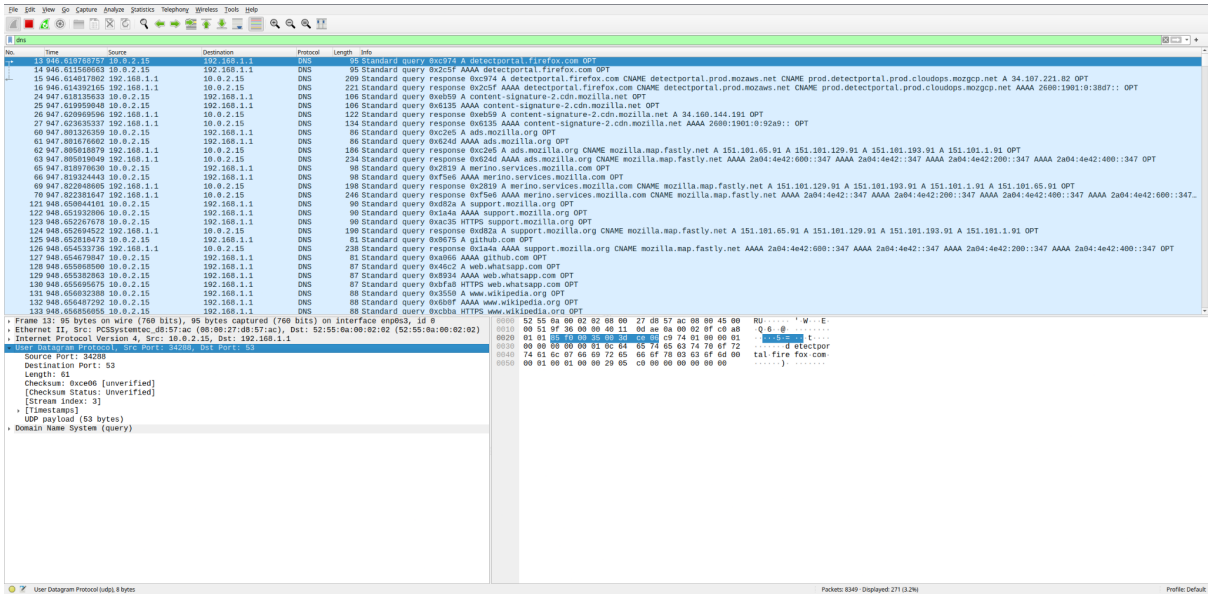
In this case, the query requests the IPv4 address (record type `A`) for the domain:

`detectportal.firefox.com`

Wireshark also provides a useful shortcut: at the bottom of the DNS section we can see a reference such as:

`[Response In: 15]`

This indicates that the corresponding DNS response appears in packet number 15. By clicking on this reference, Wireshark automatically jumps to the packet containing the reply.



If we expand the **User Datagram Protocol (UDP)** section, Wireshark shows the fields of the UDP header:

- **Source Port**
- **Destination Port**
- **Length**
- **Checksum**

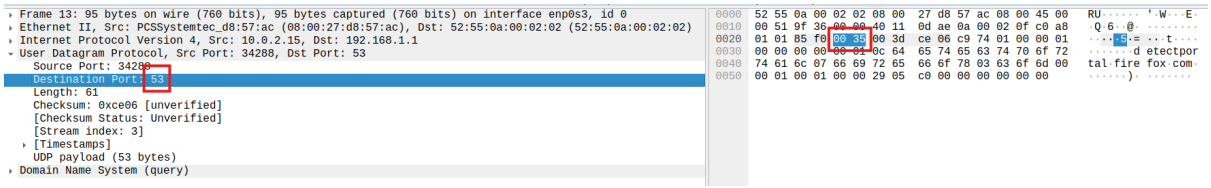
For example, in the selected packet we observe:

- Source Port: 34288
- Destination Port: 53

The destination port **53** indicates that the packet is directed to a DNS server.

Recall of Hexadecimal Representation

DECIMAL	HEX	BINARY
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111



In the lower-right panel, Wireshark shows the raw packet bytes in **hexadecimal format**. Each field highlighted in the protocol dissection corresponds to specific bytes in this hexadecimal view.

For instance, if a value such as `4F` appears in the hexadecimal representation, it corresponds to the decimal value:

$$4F_{16} = 4 \times 16 + 15 = 79$$

Another example:

$$EC01_{16} = 14 \times 16^3 + 12 \times 16^2 + 0 \times 16^1 + 1 \times 16^0$$

This conversion is useful when interpreting numeric values extracted directly from packet bytes. This explains why a value such as `35` in hexadecimal actually corresponds to:

$$3 \times 16 + 5 = 53$$

which represents the DNS destination port.

3.2.4 tcpdump

Another very important tool for packet capture is **tcpdump**. Unlike Wireshark, tcpdump is a **command-line tool** and is mainly used to capture packets rather than to perform detailed interactive inspection.
tcpdump is widely used in practice because it can run on systems that do not have a graphical user interface, such as servers. In many real-world scenarios, administrators access machines remotely through the command line, and therefore tcpdump becomes an essential tool for network analysis.
For example, tcpdump can be used to capture packets and save them directly into a capture file:

```
sudo tcpdump -w capture.pcap
```

In this case, tcpdump captures all the packets and stores them into a **PCAP file**, which can later be opened and analyzed using tools such as Wireshark (use ctrl+C to stop the capture).

Tcpdump Filtering

Like Wireshark, tcpdump uses the **Berkeley Packet Filter (BPF)** syntax to define packet filters. BPF filters allow us to select only the packets that match certain conditions. You can use:

```
man pcap-filter
```

to view all available options.

Some of the most common filtering criteria include:

- **host**: captures traffic involving a specific IP address
- **src host**: captures traffic where the specified IP is the source
- **dst host**: captures traffic where the specified IP is the destination

For example:

```
tcpdump host 192.168.1.1
```

captures all traffic involving the host `192.168.1.1`. Similarly:

```
tcpdump src host 192.168.1.1
```

captures only packets where that host is the source.

It is also possible to filter traffic based on **port numbers**. For example:

```
tcpdump port 53
```

captures traffic related to port 53, which typically corresponds to DNS traffic.
Port ranges can also be specified if needed.

tcpdump can also filter packets based on the **network protocol**. For instance:

```
tcpdump ip
tcpdump ipv6
tcpdump icmp
```

These filters capture packets belonging to specific protocols.

Filters can be combined using **logical operators** such as:

- **and**
- **or**
- **not**

For example:

```
tcpdump not host 192.168.1.1
```

captures all packets except those involving the specified host.
This can be useful when trying to exclude noisy traffic generated by a particular service (for example a video call or streaming application).

By default, tcpdump tries to resolve IP addresses into hostnames. For example, instead of displaying an IP address, it may show a domain name.
If we want to disable this behavior and display only numerical IP addresses, we can use the `-n` option:

```
tcpdump -n
```

This option avoids DNS resolution and makes the output easier to read, especially during real-time packet capture.